

Final Project

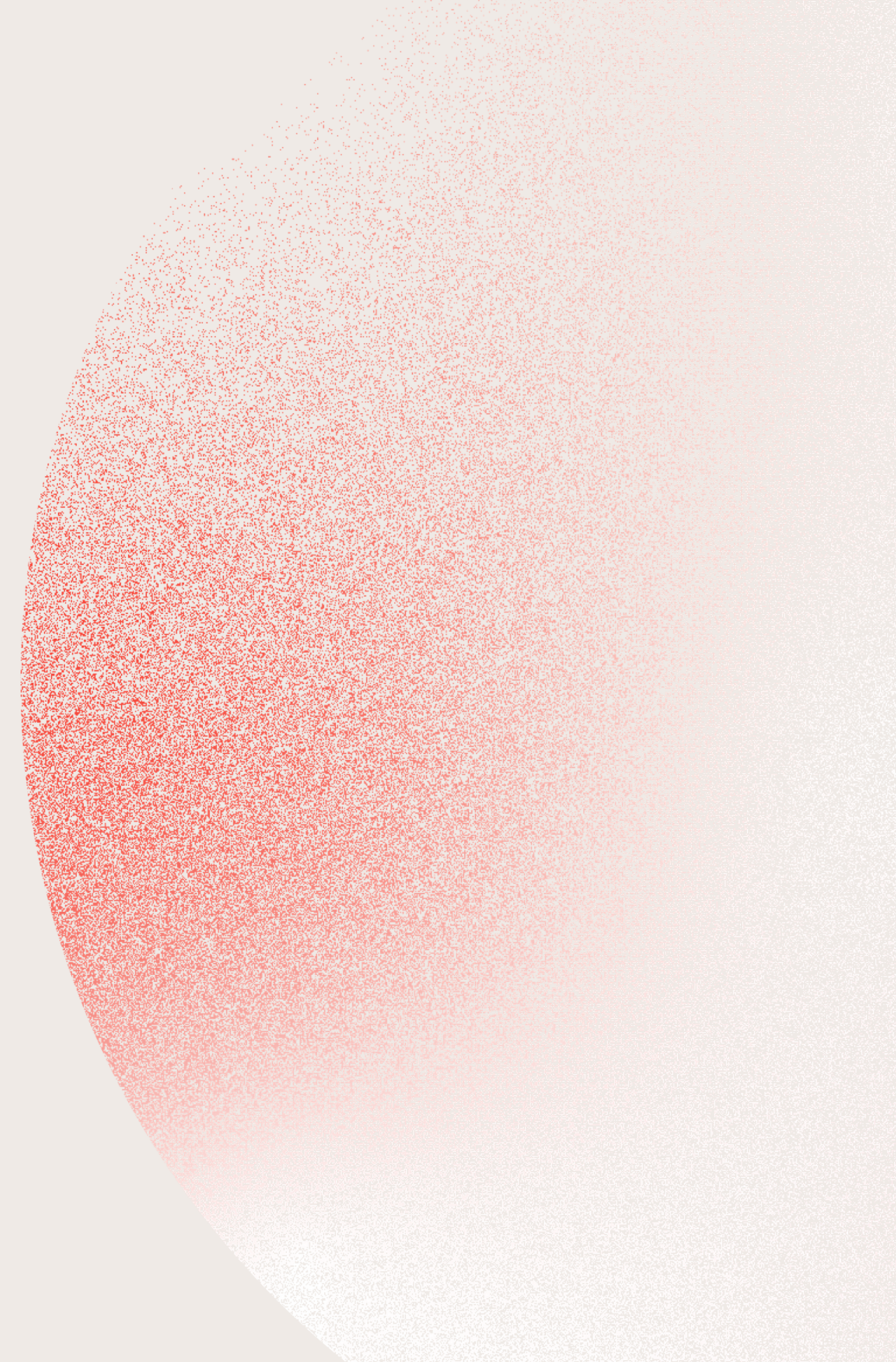
Physics-informed Neural Networks (PINNs)
for computational solid mechanics

Team: Group 4

Members: 土木116_黃鈺普_E64124137

土木116_韓昀倫_E64139530

Course: 114-2 Computer Programming



Project Statement

- Evaluates PINNs using distinct governing equations
- Forward Problem: Predicts structural deflection given material properties and loading conditions.
- Inverse Problem: Discovers the material's Young's modulus using sparse, measured deflection data.
- Incorporates the Deep Energy Method (DEM) to resolve the weight imbalance and gradient conflict issues common in standard PINN loss functions.
- Compares results with analytical solutions from Euler-Bernoulli and Timoshenko beam theories.

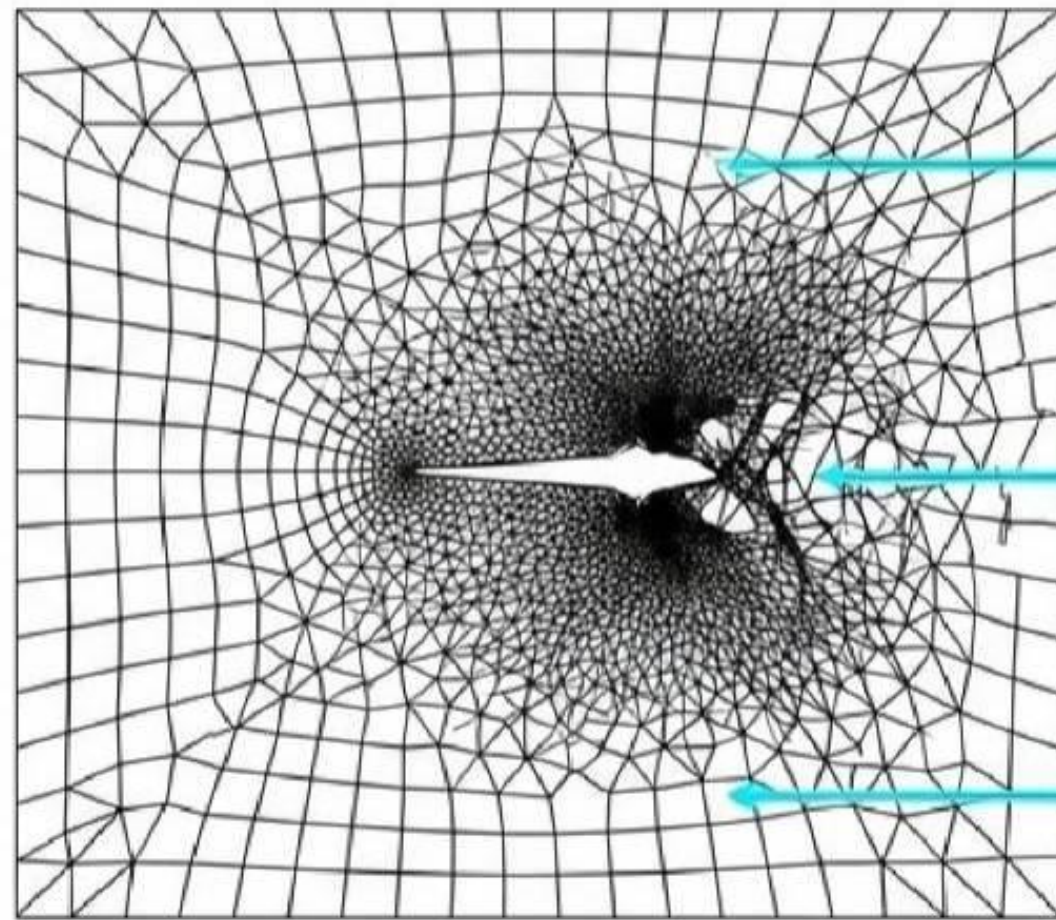
Table of Contents

- 01 . Research Background
- 02 . Introduction to Artificial Neural Network
- 03 . Mechanics Review
- 04 . Numerical Example
- 05 . Conclusion
- 06 . Appendix
- 07 . Reference

Part 1 : Research Background

1.1 Limitation of Traditional Method

THE BOTTLENECK OF FINITE ELEMENT METHOD (FEM)



FEM is computationally expensive. Complex geometries and material defects lead to severe mesh-generation costs and simulation failures.

The engineering paradox:
We need a meshfree method that is physically incapable of breaking the laws of mechanics.

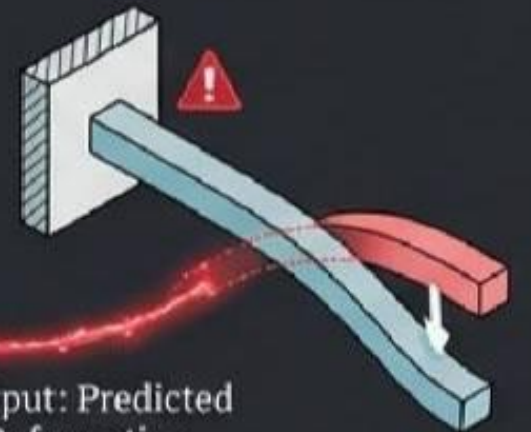
THE DANGER OF PURE MACHINE LEARNING (ML)

Input Data:
Load, Geometry,
Material

Training Data:
 10^6 Samples

BLACK BOX
ML MODEL

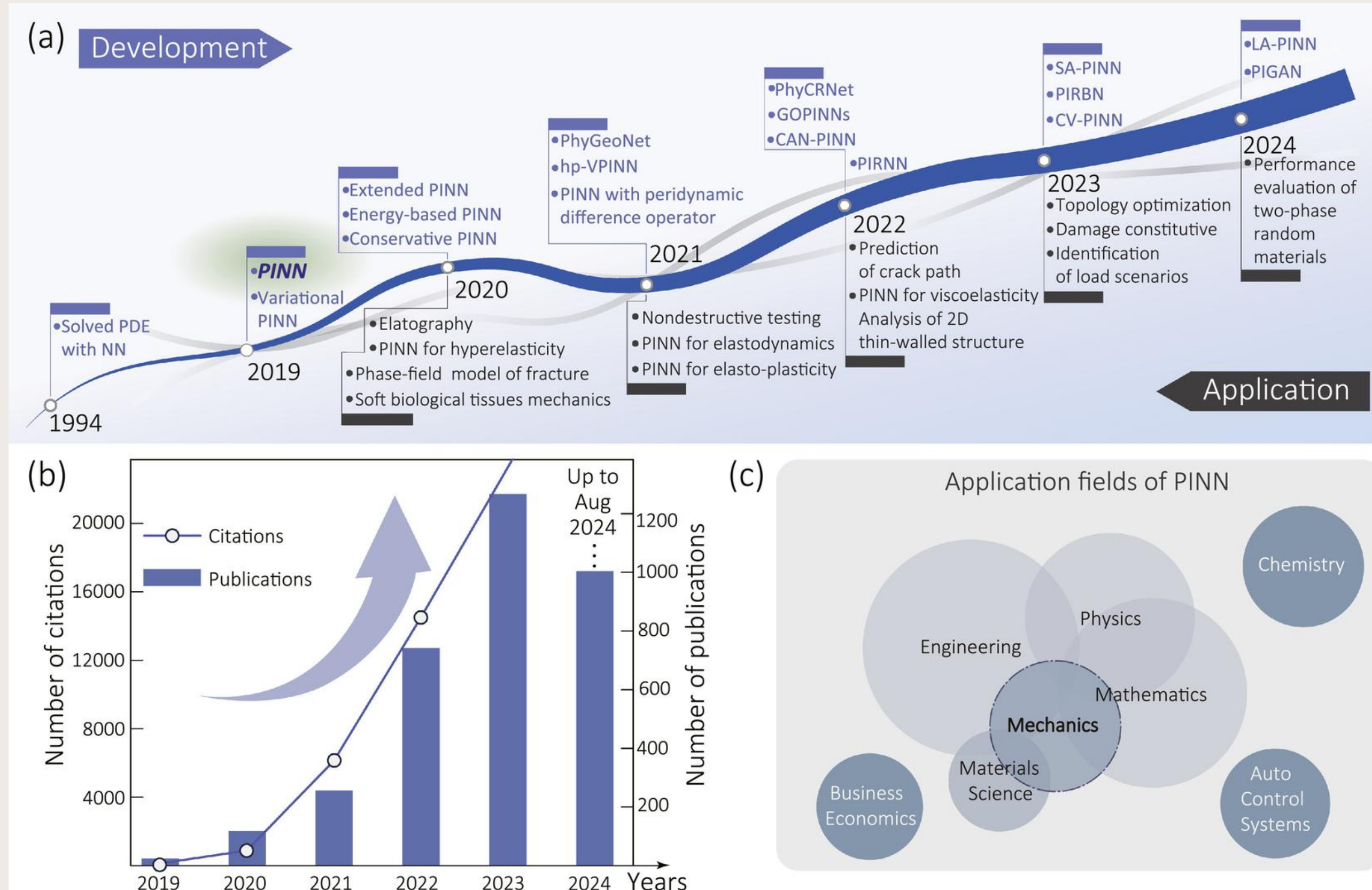
Output: Predicted
Deformation



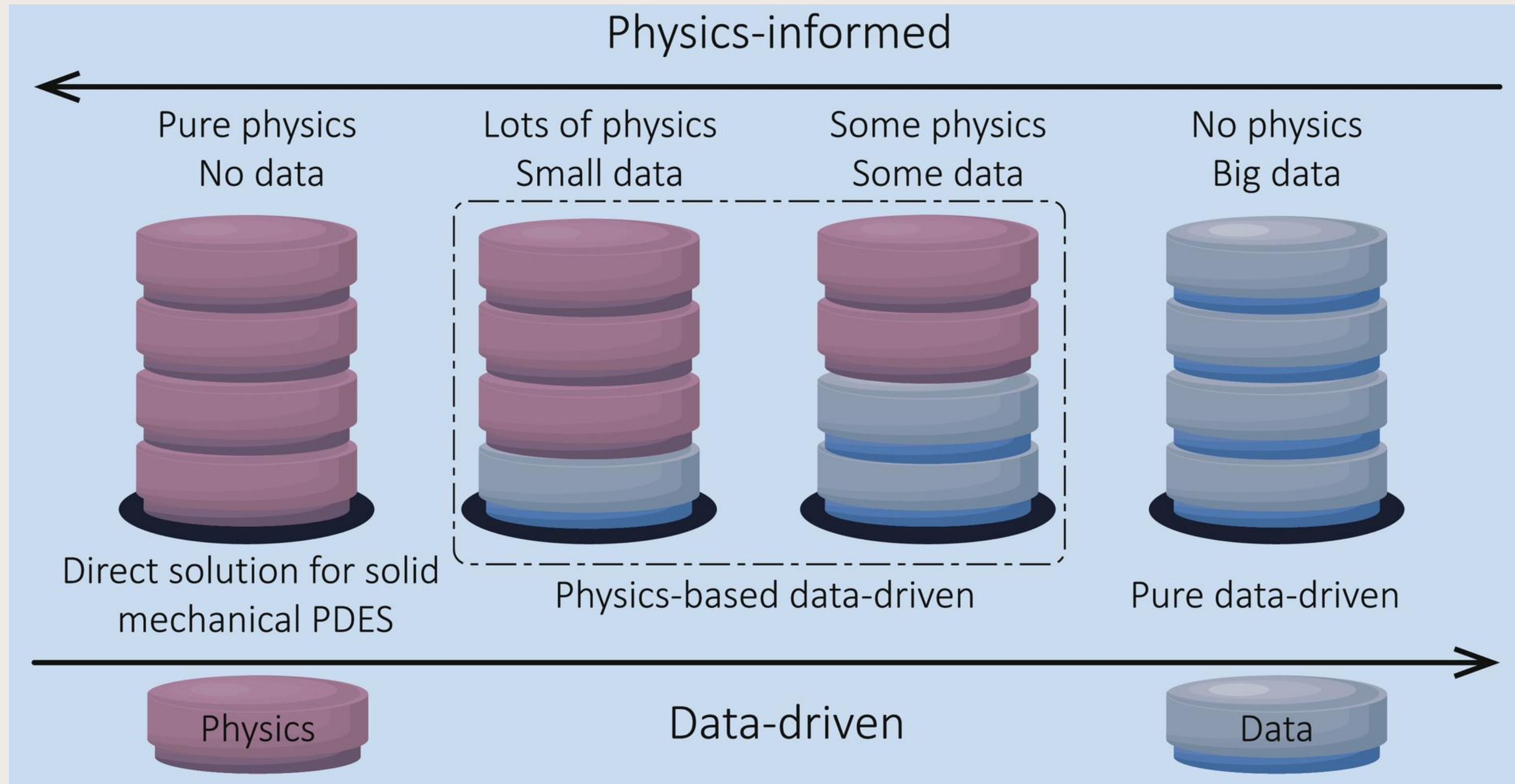
Data-driven deep learning fits curves beautifully but actively violates fundamental physics, posing severe safety risks in civil engineering.

Part 2 : Introduction to Artificial Neural Network

2.1 Introduction to PINNs

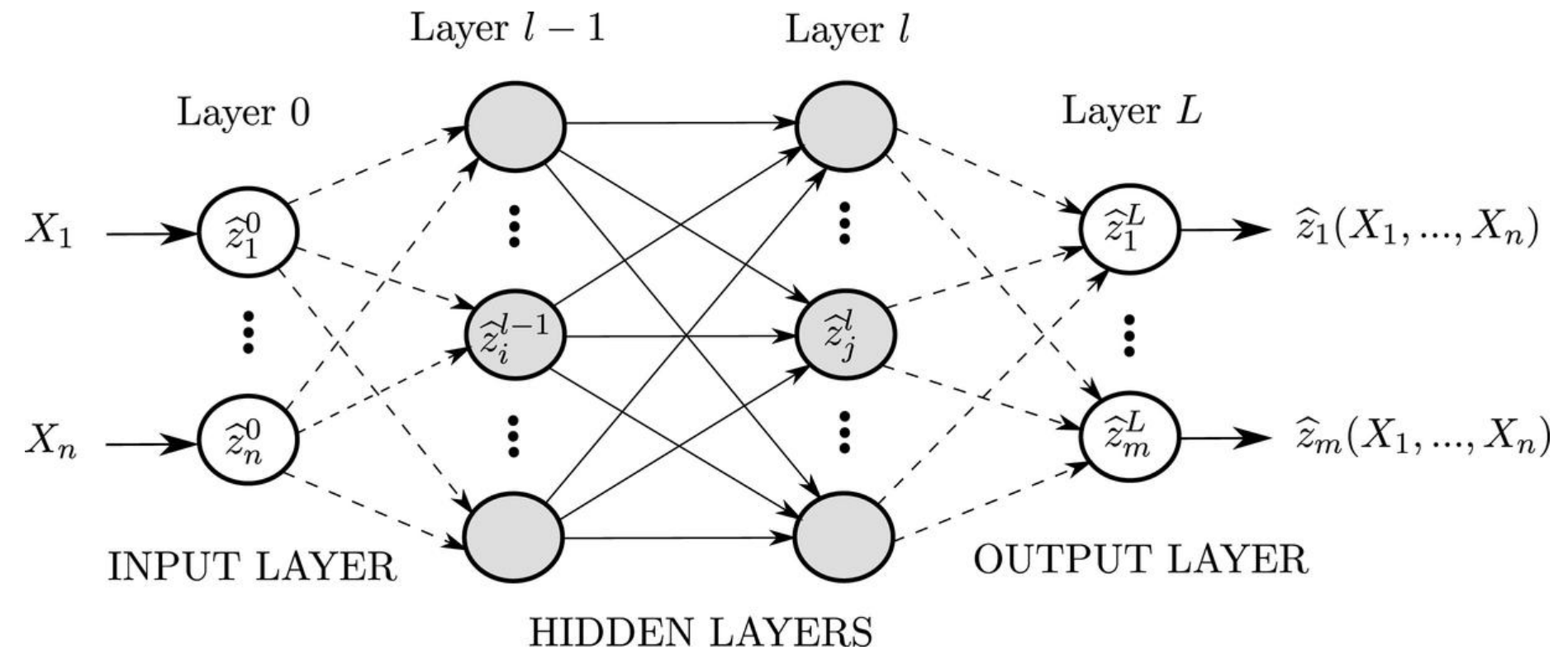


2.2 Features & Advantages of PINNs



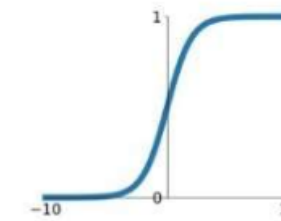
2.3.1 Neural Network(NN)

- Input Layer
- Hidden Layer
- Output Layer
- Activation Functions



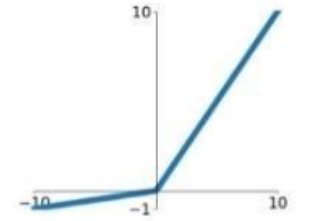
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



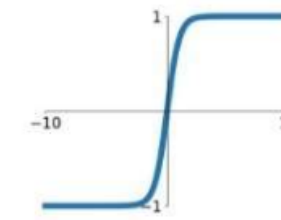
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

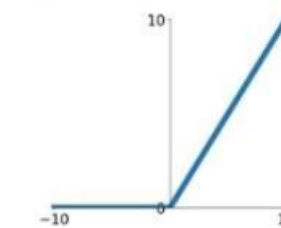


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

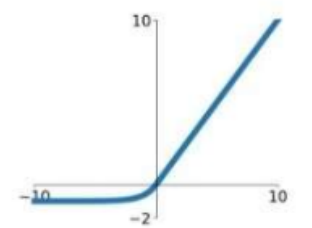
ReLU

$$\max(0, x)$$



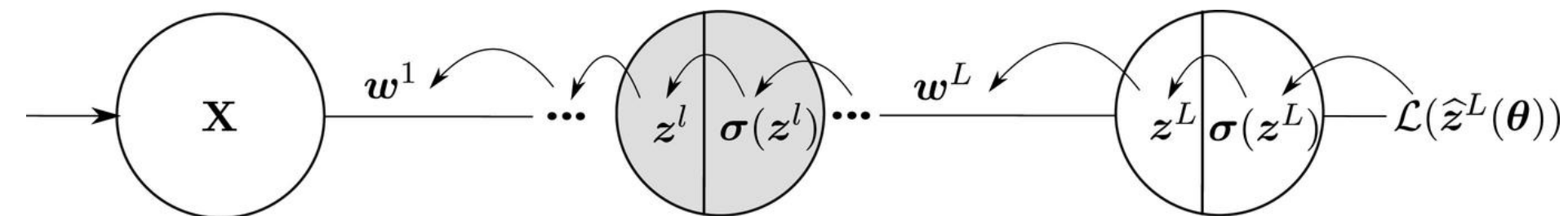
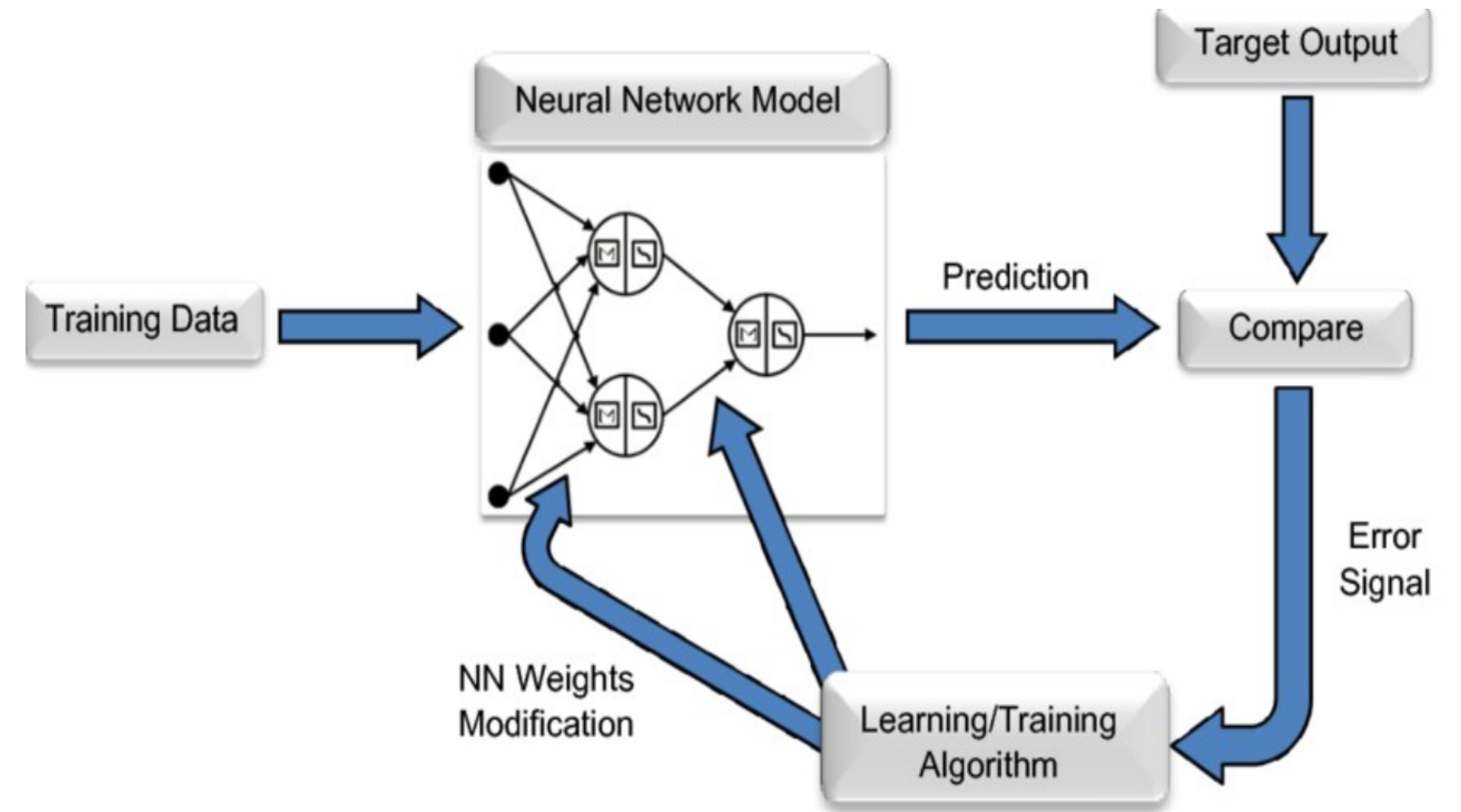
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



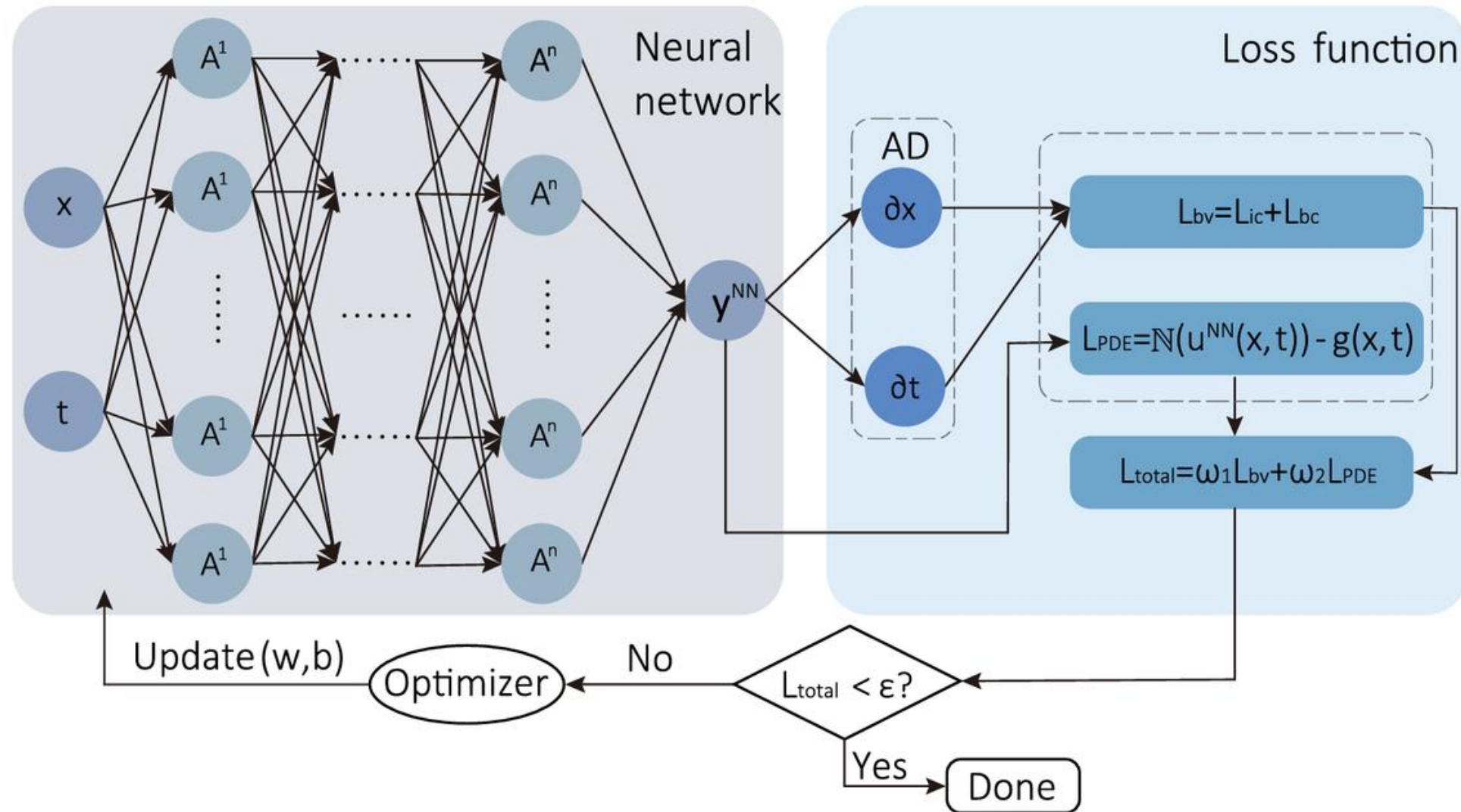
2.3.1 Neural Network(NN)

- Auto Differentiation
- Backward Propagation
- Optimizer(Adam, L-BFGS)
- Weights, w
- Biase, b



$$MSE = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}$$
$$W = W - \alpha \frac{\partial L}{\partial W}$$
$$b = b - \alpha \frac{\partial L}{\partial b} \quad (\alpha - \text{Learning Rate})$$

2.3.2 Physics Informed Neural Networks(PINNs)



$$\vartheta \frac{\partial^2 u}{\partial x^2} = \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x}$$

$$u(x, 0) = \sin(x), x \in [-1, 1]$$

$$u(\pm L, t) = 0, L \in [0, 1]$$

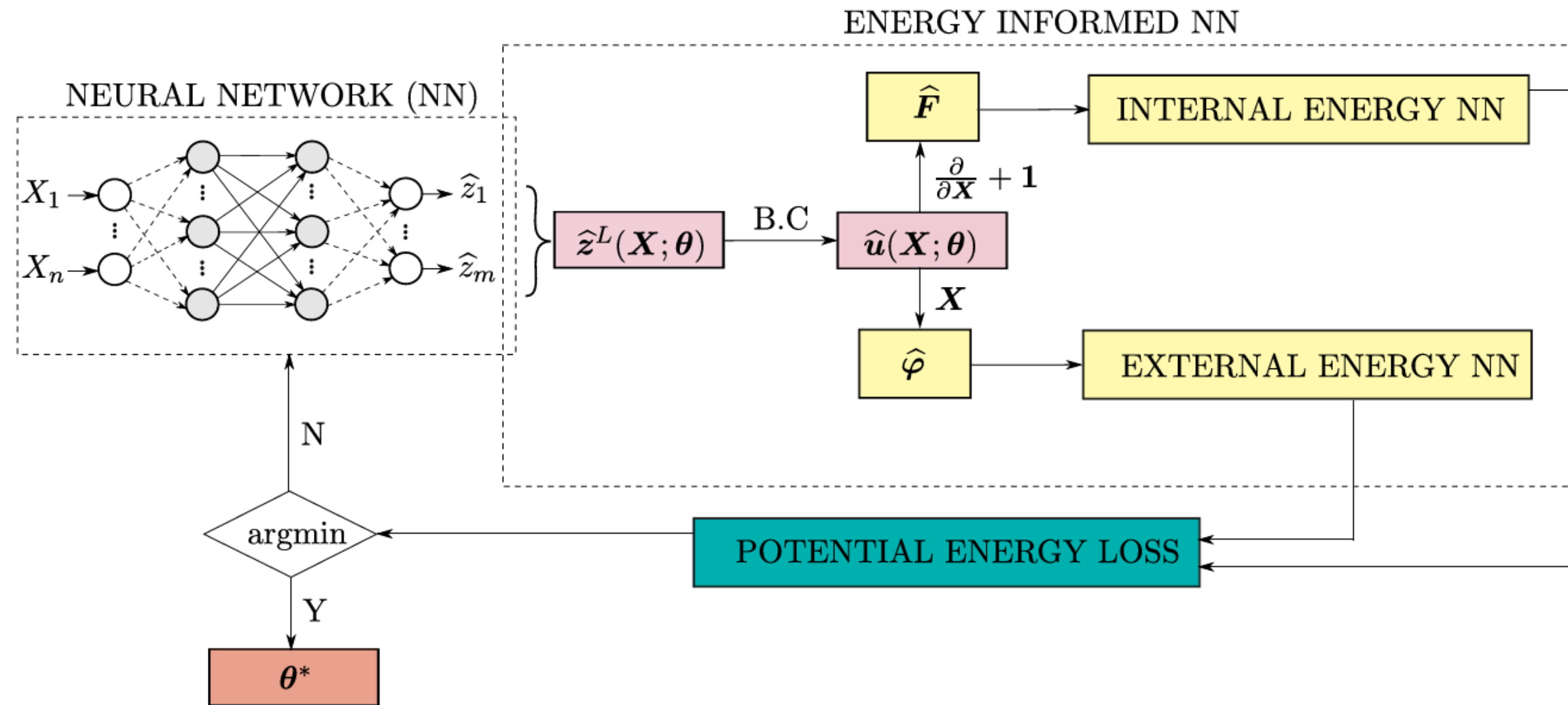
$$R = \vartheta \frac{\partial^2 u}{\partial x^2} - \frac{\partial u}{\partial t} - u \frac{\partial u}{\partial x}$$

$$MSE = \left(\vartheta \frac{\partial^2 u}{\partial x^2} - \frac{\partial u}{\partial t} - u \frac{\partial u}{\partial x} \right)^2 + (u(x, 0) - \sin(x))^2 + (u(\pm L, t) - 0)^2$$

$$MSE = MSE_R + MSE_{[BC, IC]}$$

$$\text{Loss}_{total} = \omega_f \text{Loss}_{physics} + \omega_b \text{Loss}_{boundary}$$

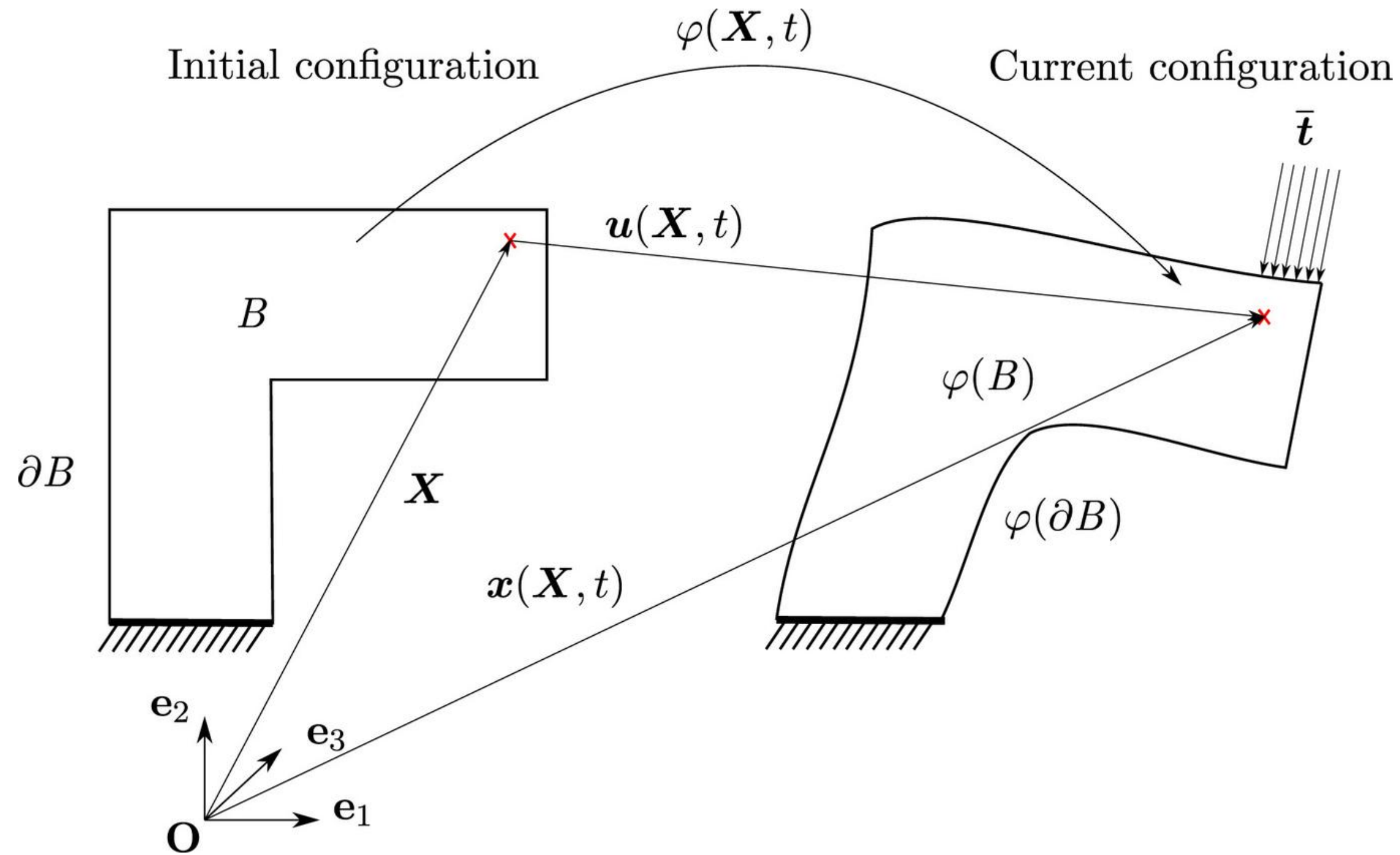
2.3.3 Deep Energy Method (DEM)



$$\Pi = H_{in} - H_{ext},$$

Part 3 : Mechanics Review

Relation between Stress & Strain



Governing Equations

$$1. \frac{d}{dx} \left(E \frac{du}{dx} \right) = 0, \quad \frac{d\hat{p}}{dx} = 0$$

$$2. \frac{d^2 u}{dx^2} - \frac{M(x)}{EI} = 0$$

$$3. EI \frac{d^4 w}{dx^4} = q$$

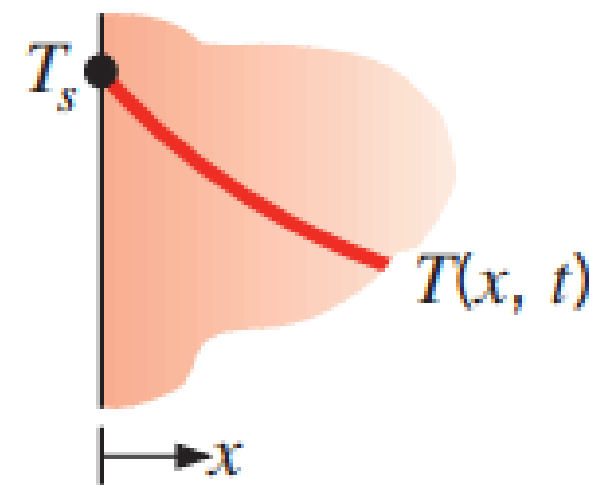
$$4. \pi(u) = \mathcal{U}(u) - \mathcal{W}(u) \rightarrow \text{Minimize}$$

$$5. U_{\text{density}} = \frac{1}{2} \lambda_{ps} (\epsilon_{xx} + \epsilon_{yy})^2 + \mu (\epsilon_{xx}^2 + \epsilon_{yy}^2 + 2\epsilon_{xy}^2)$$

Boundary Conditions

1. Constant surface temperature

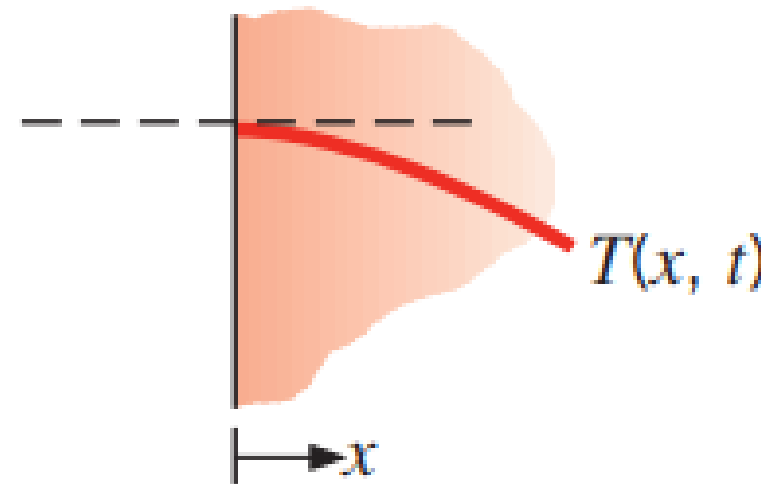
$$T(0, t) = T_s$$



Dirichlet condition

2. Adiabatic or insulated surface

$$\left. \frac{\partial T}{\partial x} \right|_{x=0} = 0$$



Neumann condition

Part 4 : Numerical Examples

4-1. 一維桿件受軸力作用

4-2. 二維簡支梁受均佈載重

4-3. 二維懸臂梁受均佈載重

4-4. 反向問題(例子:二維懸臂梁受均載)

4-5. 深度能量法討論(例子:二維懸臂梁受均載)

4-1 : 1D Cantilever Beam with axial force



材料參數與模型參數

$L = 10.0$ (樑長度, 單位: mm)

$E = 1000.0$ (楊氏模數, 單位: MPa)

$\bar{T} = 10.0$ (右端邊界拉力, 單位: MPa)

$N_{points} = 61$ (訓練點分佈個數)

$lr = 0.001$ (學習率)

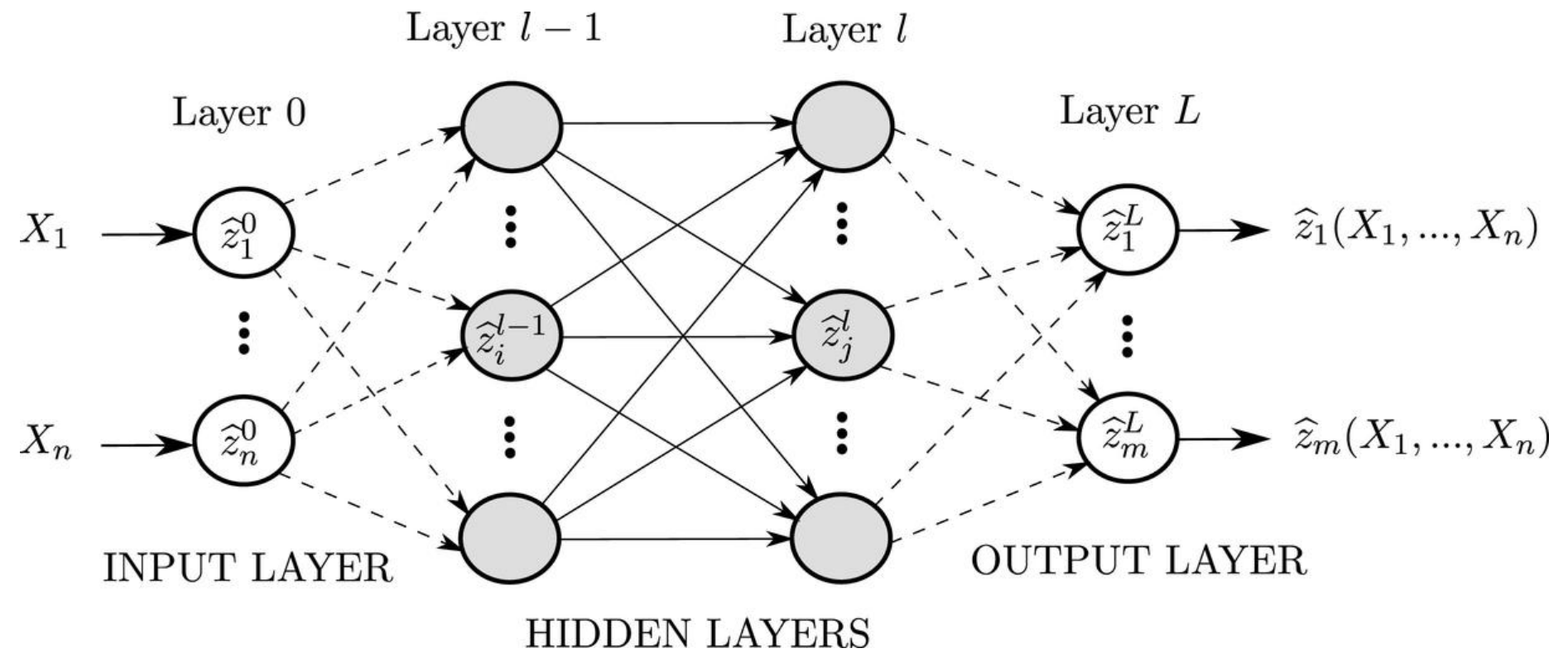
方程式

$$\text{Governing Differential Equation : } \frac{d}{dx} \left(E \frac{du}{dx} \right) = 0, \quad \frac{d\hat{P}}{dx} = 0$$

$$\text{Boundary Conditions : } u(0) = 0, \quad E \frac{du}{dx} \Big|_{x=L} = \bar{T}$$

$$\text{Loss Function : } MSE = \left(\frac{d}{dx} \left(E \frac{du}{dx} \right) \right)^2 + \left(\frac{d\hat{P}}{dx} \right)^2 + 100 \left(E \frac{du}{dx} - \hat{P} \right)^2 + \left(E \frac{du}{dx} \Big|_{x=L} - \bar{T} \right)^2$$

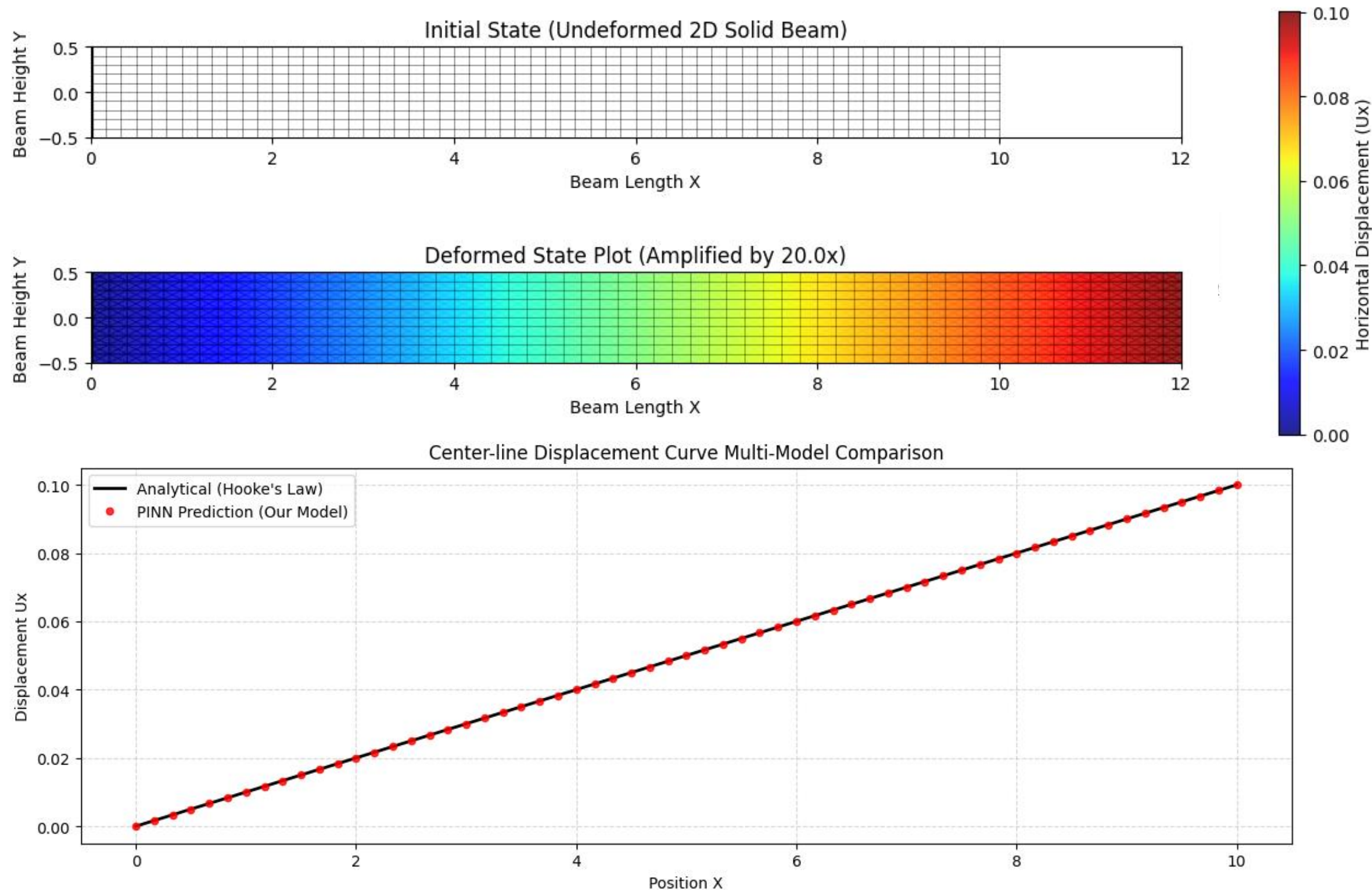
4-1 : 1D Cantilever Beam with axial force-PINN



- 輸入層：1層
- 隱藏層：5層
- 輸出層：1層
- 激活函數： $\tanh(x)$

```
31 class PINN(nn.Module):
32     def __init__(self):
33         super(PINN, self).__init__()
34         layers = [nn.Linear(1, 60), nn.Tanh()] # 輸入層 -> 第一隱藏層
35         for _ in range(5): # 中間的五個隱藏層
36             layers.append(nn.Linear(60, 60)) # 隱藏層各60個神經元
37             layers.append(nn.Tanh()) # 激活函數: tanh()
38         layers.append(nn.Linear(60, 2)) # 輸出層: 輸出 u_hat 與 P_hat
39         self.net = nn.Sequential(*layers)
```

4-1 : 1D Cantilever Beam with axial force-Result



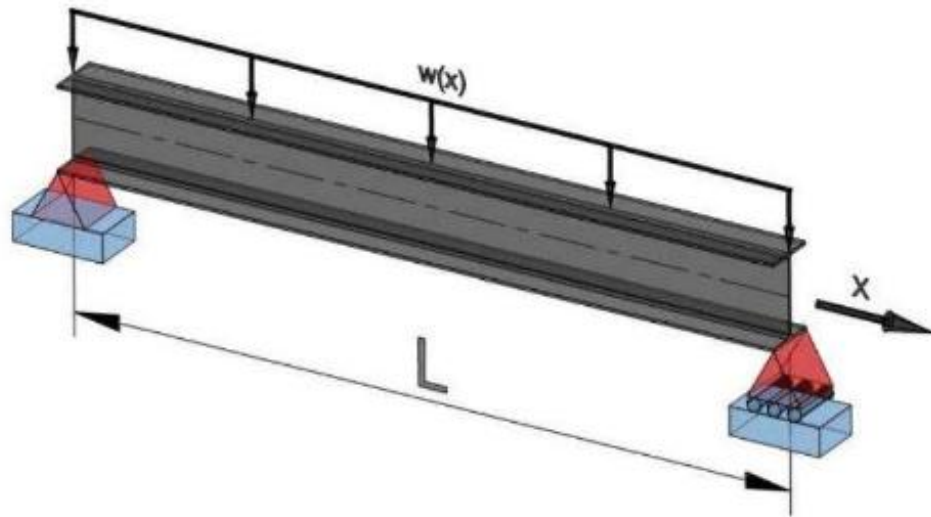
目前使用的運算裝置: cuda

開始訓練 PINN 模型...

Epoch	0 / 65000	Loss: 3.6593e+04
Epoch	5000 / 65000	Loss: 3.4598e-02
Epoch	10000 / 65000	Loss: 4.0818e-02
Epoch	15000 / 65000	Loss: 2.1547e-02
Epoch	20000 / 65000	Loss: 1.3764e-01
Epoch	25000 / 65000	Loss: 8.2903e-04
Epoch	30000 / 65000	Loss: 3.6758e-01
Epoch	35000 / 65000	Loss: 2.3311e-03
Epoch	40000 / 65000	Loss: 4.2513e-01
Epoch	45000 / 65000	Loss: 6.5206e-05
Epoch	50000 / 65000	Loss: 1.4349e-02
Epoch	55000 / 65000	Loss: 2.6912e-02
Epoch	60000 / 65000	Loss: 2.7097e-04
Epoch	65000 / 65000	Loss: 9.9876e-04

訓練完成!

4-2 : 2D Simply Supported Beam with UDL



Governing Equation : $\frac{d^2 u}{dx^2} - \frac{M(x)}{EI} = 0$

Loss Function: Total Loss = Loss_{pDE} + λ_{bc} * Loss_{bc}

Boundary Conditions : $u(0) = 0$

$$u(10) = 0$$

$$u'(0) = \theta_0 = \frac{wL^3}{24EI}$$

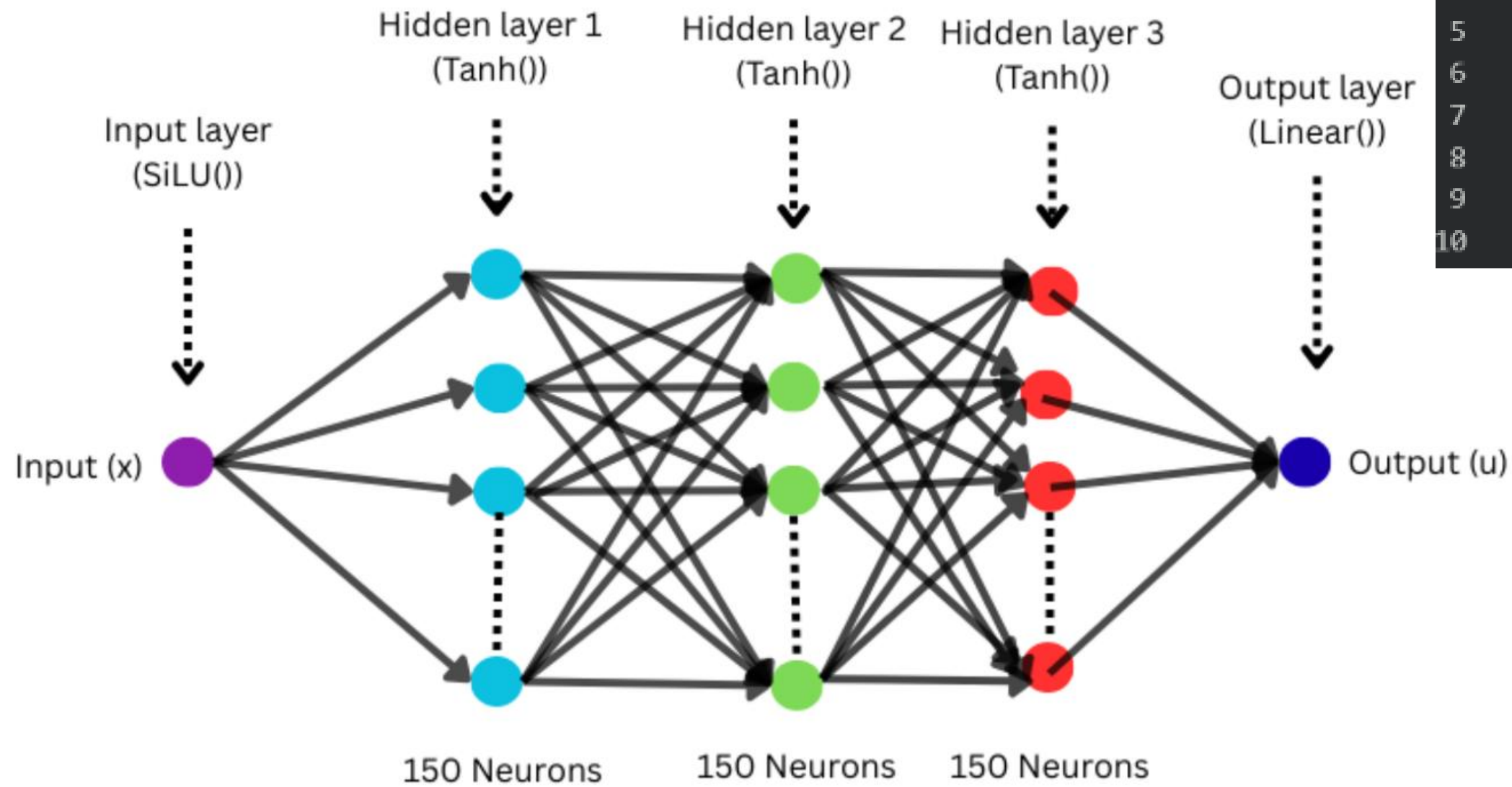
$$u'(5) = 0$$

$$u'(10) = -\theta_0 = -\frac{wL^3}{24EI}$$

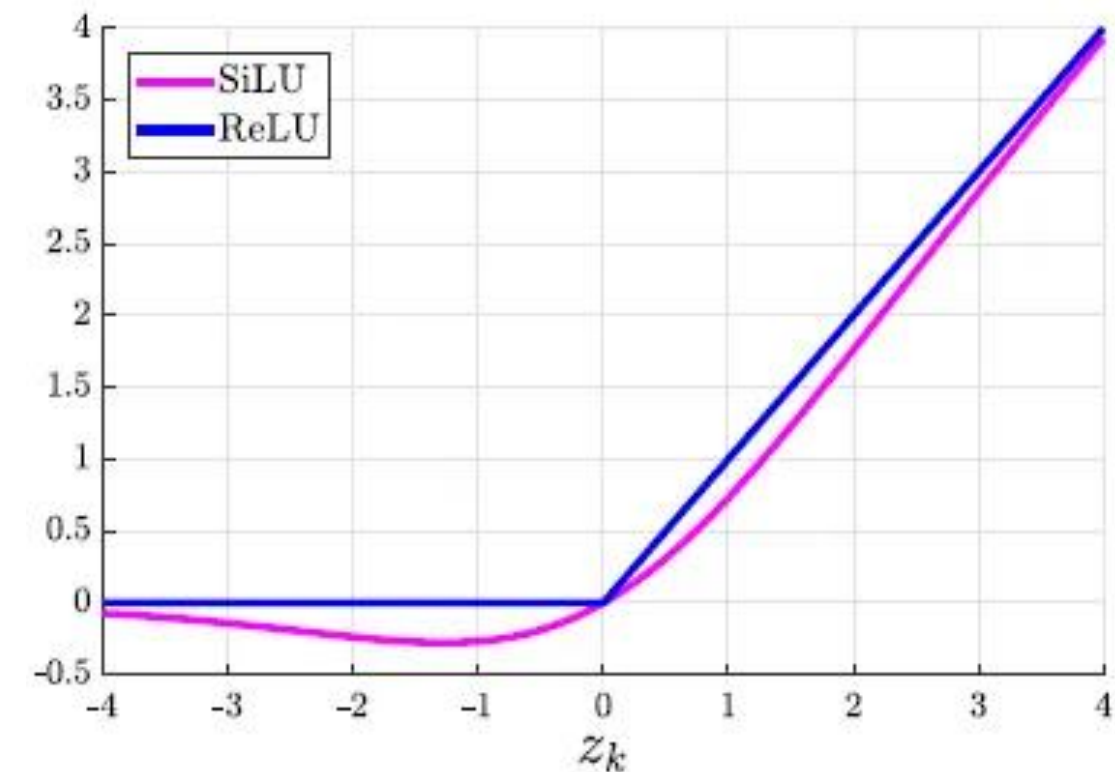
材料參數與模型參數	
Beam Length (L)	10.0m
UDL (w)	4.0kN/m
Rigidity (EI)	625000 kN*m
Collocation Points	2000
Learning Rate	初始學習率 : 0.001
	排程更新:若總損失在連續 1000 代內沒有下降, 學習率自動減半, 最低可降至 1e-6
Optimizer	Adam
Epochs	40000

4-2 : 2D Simply Supported Beam-PINNs

(1) Network: Fully-Connected Network

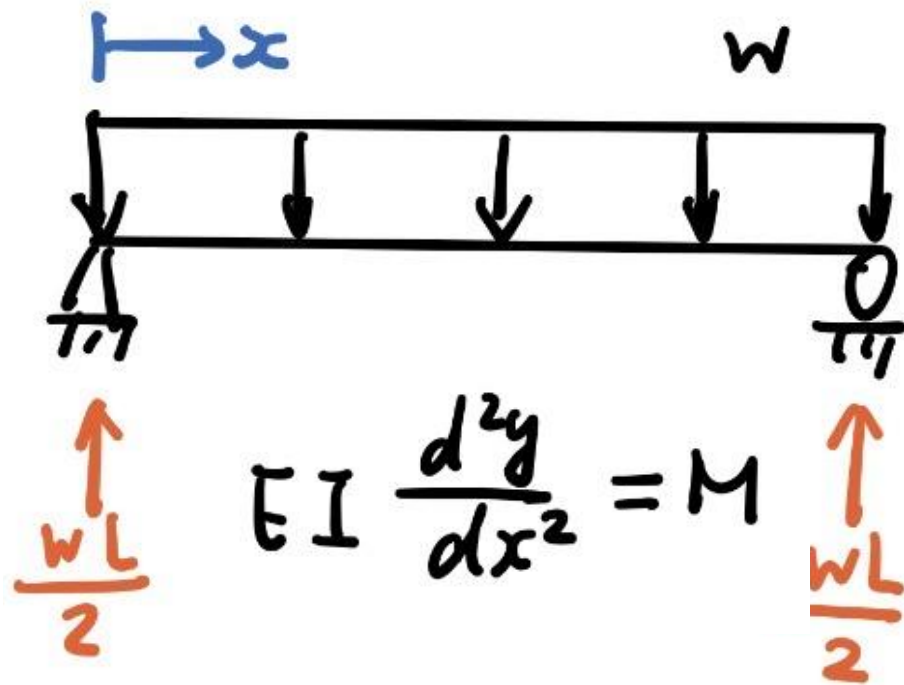


```
1 class PINN(nn.Module):
2     """Fully-connected network that approximates  $u(x)$ ."""
3
4     def __init__(self, hidden_layers: int = 4, neurons: int = 150):
5         super().__init__()
6         layers = [nn.Linear(1, neurons), nn.SiLU()]
7         for _ in range(hidden_layers - 1):
8             layers += [nn.Linear(neurons, neurons), nn.Tanh()]
9         layers.append(nn.Linear(neurons, 1))
10        self.net = nn.Sequential(*layers)
```



4-2 : 2D Simply Supported Beam-PINNs

(2)Residual



$$w(x) = -w$$

$$v(x) = \int w(x) dx = -wx + \frac{wL}{2}$$

$$M(x) = \int v(x) dx = -\frac{wx^2}{2} + \frac{wL}{2}x$$

$$\rightarrow \frac{d^2y}{dx^2} + \left(\frac{wx^2}{2} - \frac{wL}{2}x \right) / EI = 0$$

```
1 def beam_residual(model: PINN, x: torch.Tensor, w = 4.0, L = 10.0, EI = 625e3)
2     """
3     ODE: u''(x) + u(x) = 0
4     Compute residual using automatic differentiation.
5     """
6     x = x.requires_grad_(True)
7     u = model(x)
8
9     # First derivative
10    u_x = torch.autograd.grad(
11        u, x,
12        grad_outputs=torch.ones_like(u),
13        create_graph=True,
14        retain_graph=True
15    )[0]
16
17    # Second derivative
18    u_xx = torch.autograd.grad(
19        u_x, x,
20        grad_outputs=torch.ones_like(u_x),
21        create_graph=True,
22        retain_graph=True
23    )[0]
24
25    residual = u_xx + ((w * L * 0.5) * x - (w * 0.5) * (x**2)) / (625e3)
26    return residual
```

4-2 : 2D Simply Supported Beam-PINNs

(3)Boundary Condition

$$u(0)=0$$

$$u(10)=0$$

$$u'(0)=\theta_0=\frac{wL^3}{24EI}$$

$$u'(5)=0$$

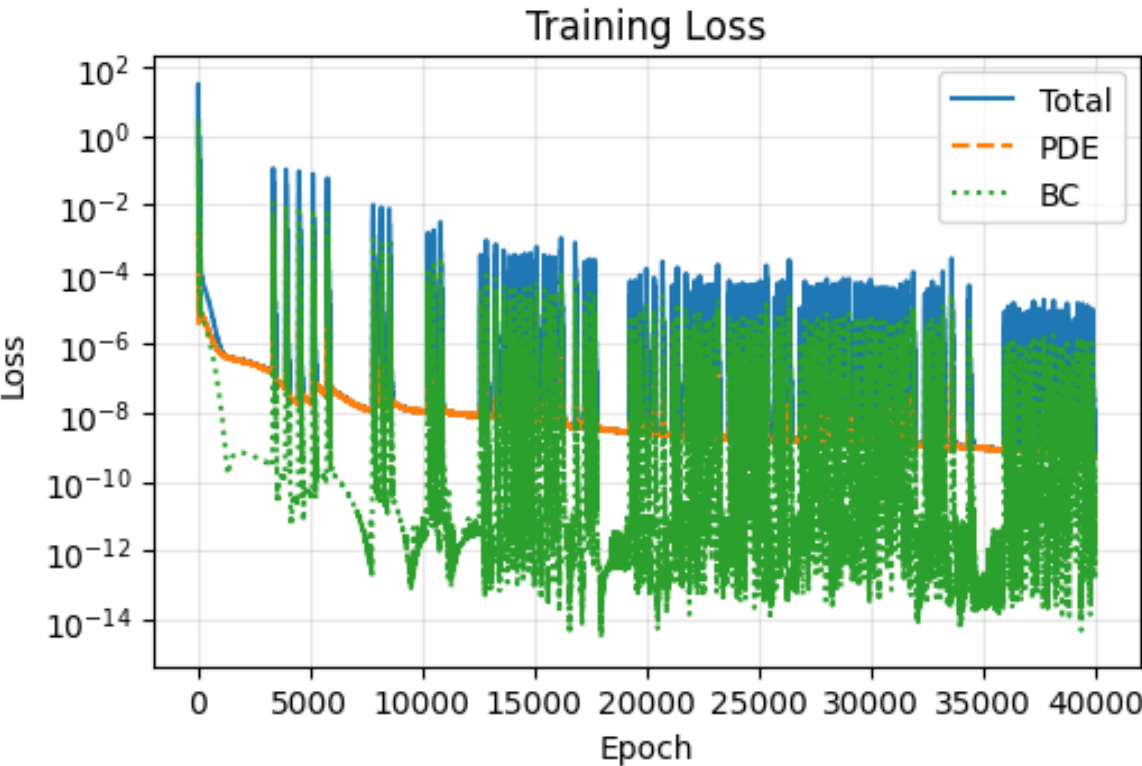
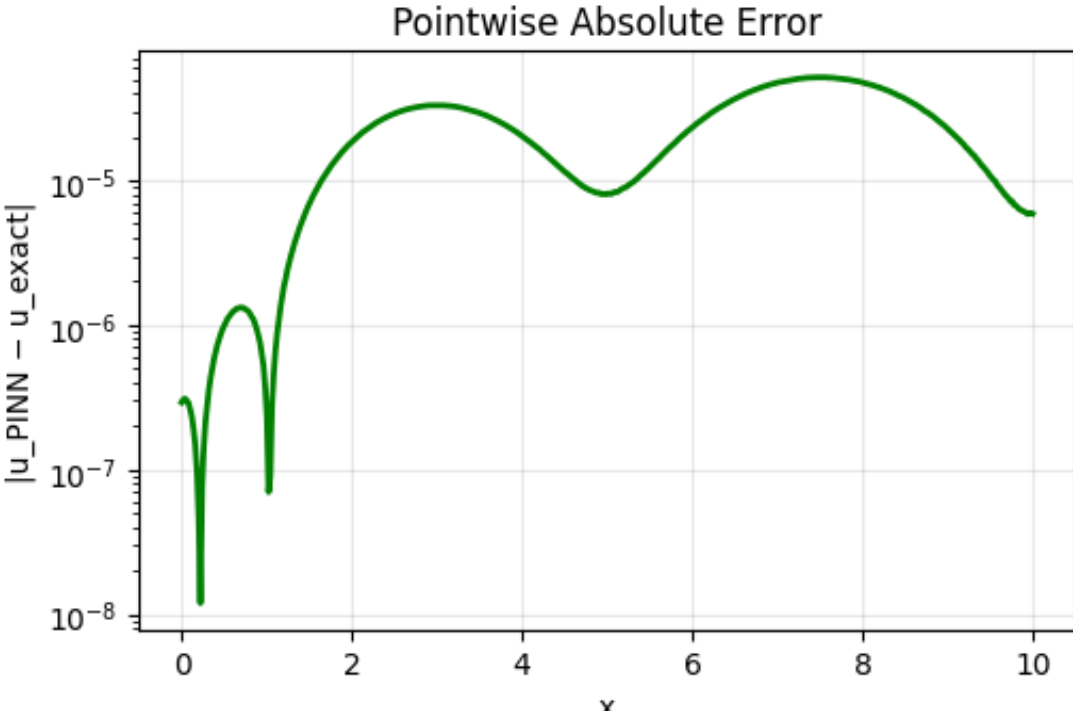
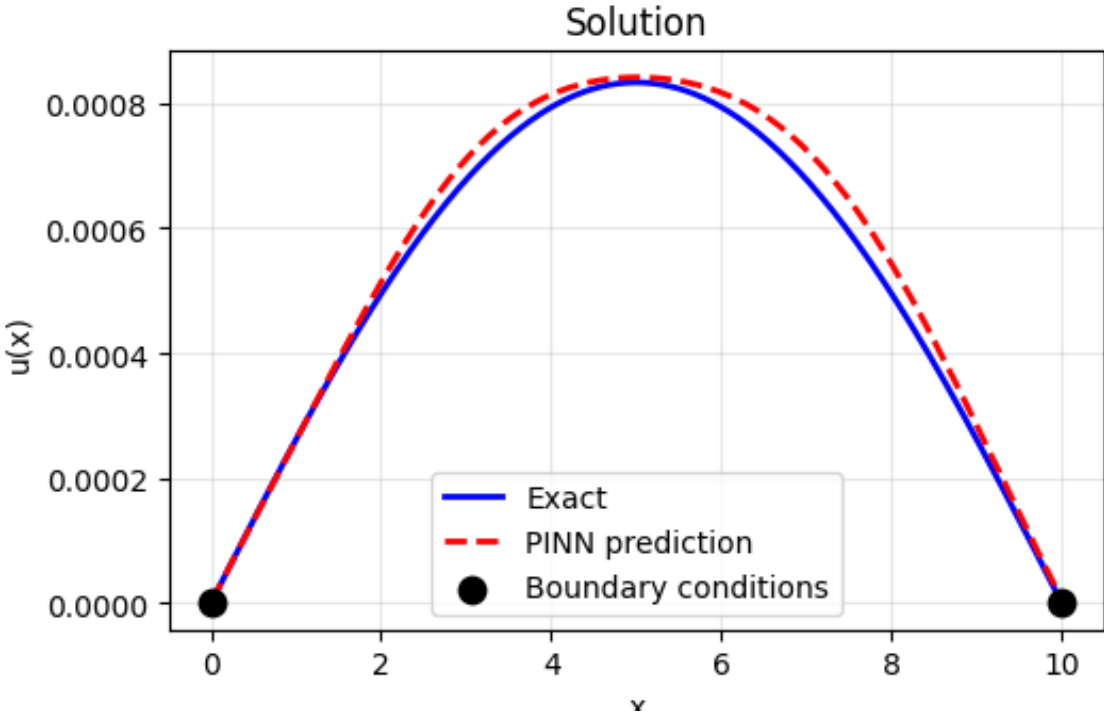
$$u'(10)=-\theta_0=-\frac{wL^3}{24EI}$$

```
x_bc_1 = torch.tensor([[0.0]], dtype=torch.float32).to(device)
u_bc_1 = torch.tensor([[0.0]], dtype=torch.float32).to(device)
x_bc_2 = torch.tensor([[10.0]], dtype=torch.float32).to(device)
u_bc_2 = torch.tensor([[0.0]], dtype=torch.float32).to(device)
x_bc_3 = torch.tensor([[0.0]], dtype=torch.float32).to(device)
ux_bc_3 = torch.tensor([[+slope]], dtype=torch.float32).to(device)
x_bc_4 = torch.tensor([[5.0]], dtype=torch.float32).to(device)
ux_bc_4 = torch.tensor([[0.0]], dtype=torch.float32).to(device)
x_bc_5 = torch.tensor([[10.0]], dtype=torch.float32).to(device)
ux_bc_5 = torch.tensor([[-1) * slope]], dtype=torch.float32).to(device)
```

4-2 : 2D Simply Supported Beam with UDL-Result

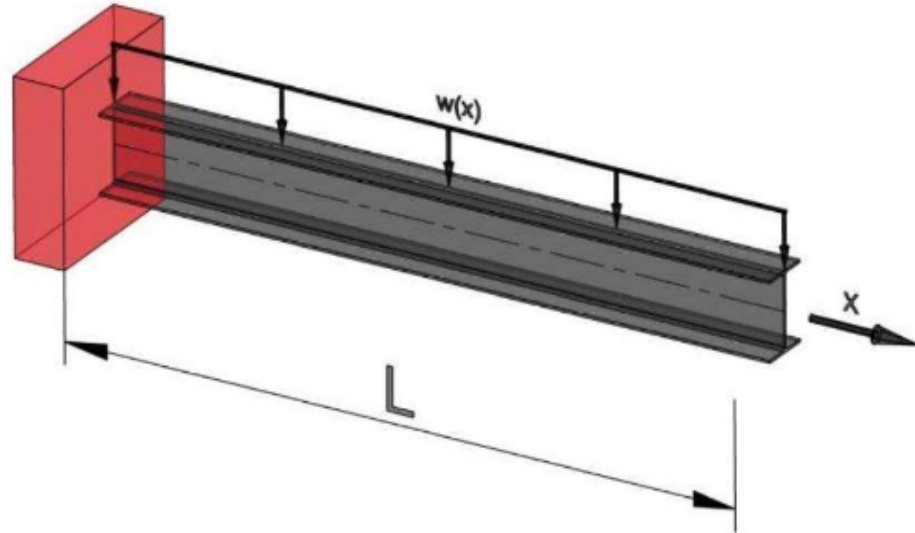
PINN Solution of $u'' - M/EI = 0$, $u(0)=u(10)=0$, $u'(5) = 0$

Epoch 1	Total: 5.18e-01	PDE: 5.64e-04	BC: 5.18e-02
Epoch 2000	Total: 3.08e-07	PDE: 3.01e-07	BC: 6.82e-10
Epoch 4000	Total: 7.95e-06	PDE: 5.86e-08	BC: 7.89e-07
Epoch 6000	Total: 4.69e-08	PDE: 4.50e-08	BC: 1.94e-10
Epoch 8000	Total: 2.12e-08	PDE: 2.07e-08	BC: 4.39e-11
Epoch 10000	Total: 1.05e-08	PDE: 1.05e-08	BC: 2.30e-12
Epoch 12000	Total: 8.70e-09	PDE: 8.67e-09	BC: 2.39e-12
Epoch 14000	Total: 8.78e-09	PDE: 6.84e-09	BC: 1.93e-10
Epoch 16000	Total: 2.96e-08	PDE: 4.88e-09	BC: 2.47e-09
Epoch 18000	Total: 3.24e-09	PDE: 3.24e-09	BC: 4.28e-14
Epoch 20000	Total: 1.15e-05	PDE: 6.85e-09	BC: 1.15e-06
Epoch 22000	Total: 1.15e-05	PDE: 3.15e-09	BC: 1.15e-06
Epoch 24000	Total: 2.36e-09	PDE: 1.78e-09	BC: 5.73e-11
Epoch 26000	Total: 1.91e-09	PDE: 1.51e-09	BC: 4.01e-11
Epoch 28000	Total: 1.33e-09	PDE: 1.32e-09	BC: 8.90e-13
Epoch 30000	Total: 1.23e-05	PDE: 2.53e-09	BC: 1.23e-06
Epoch 32000	Total: 4.41e-09	PDE: 1.26e-09	BC: 3.15e-10
Epoch 34000	Total: 1.01e-09	PDE: 1.01e-09	BC: 1.38e-14
Epoch 36000	Total: 6.00e-08	PDE: 8.30e-10	BC: 5.92e-09
Epoch 38000	Total: 4.30e-08	PDE: 7.13e-10	BC: 4.23e-09
Epoch 40000	Total: 9.01e-10	PDE: 5.66e-10	BC: 3.35e-11



Max absolute error : 5.1700e-05
Mean absolute error: 2.2512e-05

4-3 : 2D Cantilever Beam with UDL



Governing Equation : $EI \frac{d^4 w}{dx^4} = q$

with scaling (無因次化) $\rightarrow \frac{d^4 \bar{w}}{d\bar{x}^4} = 1.0$

Loss Function: Total Loss = $Loss_{pDE} + \lambda_{bc} \times Loss_{bc}$

Boundary Conditions : $\bar{w}(0) = 0$

$$\frac{d\bar{w}}{d\bar{x}}(0) = 0$$

$$\frac{d^2 \bar{w}}{d\bar{x}^2}(1) = 0$$

$$\frac{d^3 \bar{w}}{d\bar{x}^3}(1) = 0$$

材料參數與模型參數	
Beam Length (L)	1.0m
UDL (w)	10000.0 N/m
Young's Modulus (E)	210 Gpa
Collocation Points	300
Learning Rate	1. Adam : 初始 lr = 1e-3 , 降至最低 1e-5
	2. L-BFGS : lr = 1.0
Optimizer	採用兩階段混合訓練 :
	1. Adam (CosineAnnealingLR 學習率排程)
Epochs	2.L-BFGS 優化器 (strong_wolfe 線性搜索法)
	Adam 訓練 2000 代
	L-BFGS 疊代 500 次迴圈

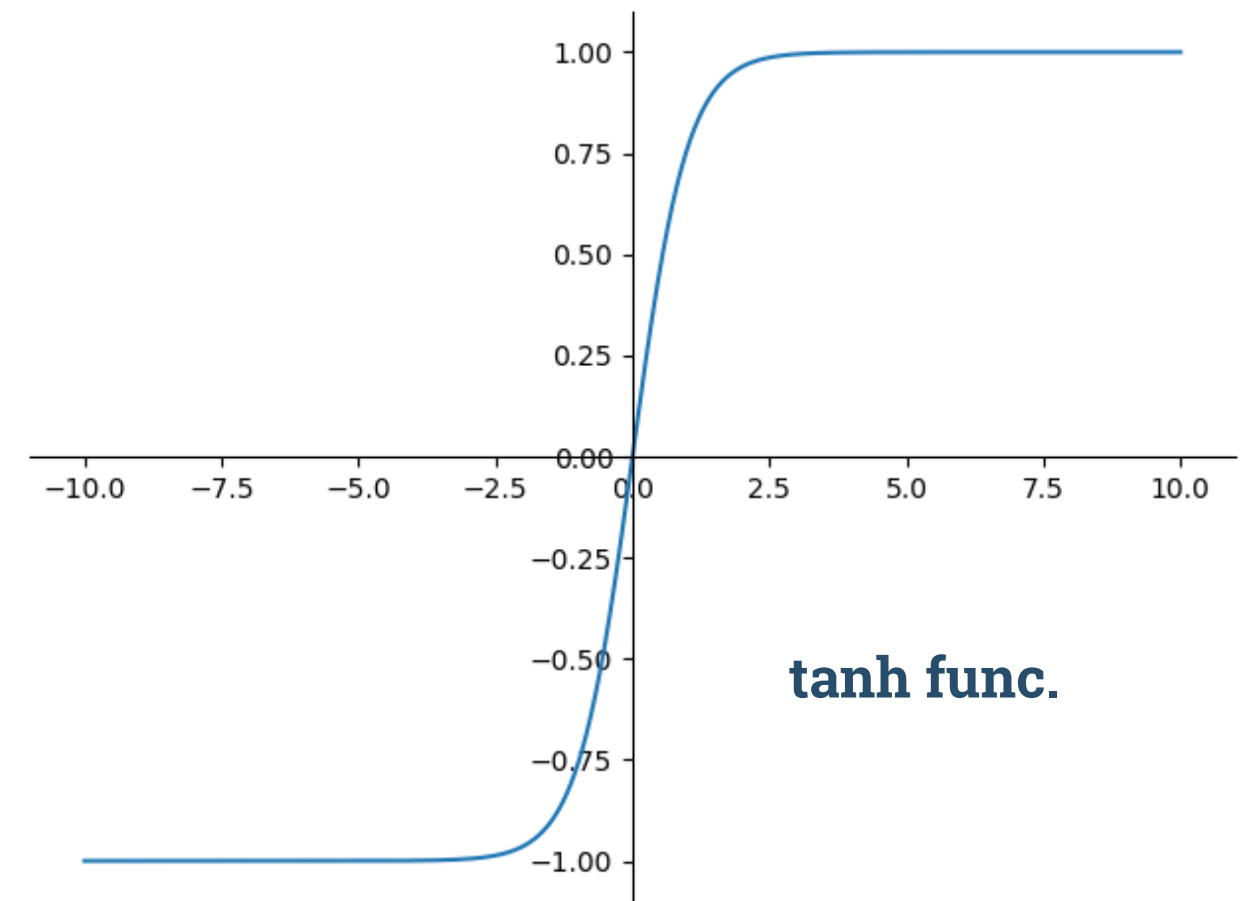
4-3 : 2D Cantilever Beam with UDL- PINN

(1)Network: Fully-Connected Network

```
class BeamPINN(nn.Module):
    def __init__(self, hidden_dims=(64, 64, 64, 64), output_dim=1):
        super().__init__()
        layers = []
        prev = 1  # 輸入層：只有一個特徵，即座標 x

        # 建立多層隱藏層，並使用 Tanh 作為激活函數（因為需要求到四階導數，必須平滑）
        for h in hidden_dims:
            layers.append(nn.Linear(prev, h))
            layers.append(nn.Tanh())
            prev = h

        # 輸出層：輸出一個值，即該點的預測位移 w
        layers.append(nn.Linear(prev, output_dim))
        self.net = nn.Sequential(*layers)
        self._init_weights() # 初始化權重
```



64 neurons * 4 layers, with **tanh()** as activation function

4-3 : 2D Cantilever Beam with UDL

(2)Optimizer(Adam & L-BFGS)

```
print("Forward PINN Training (Adam)...")
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=n_adam, eta_min=1e-5)

for epoch in range(n_adam):
    optimizer.zero_grad()
    loss, lpde, lbc = forward_loss(model, x_bar, lambda_bc)
    loss.backward()
    optimizer.step()
    scheduler.step()

    loss_history["total"].append(loss.item())
    loss_history["pde"].append(lpde.item())
    loss_history["bc"].append(lbc.item())
```

```
print("Forward PINN Training (L-BFGS)...")
optimizer2 = torch.optim.LBFGS(
    model.parameters(), lr=1.0, max_iter=20,
    history_size=50, line_search_fn="strong_wolfe"
)

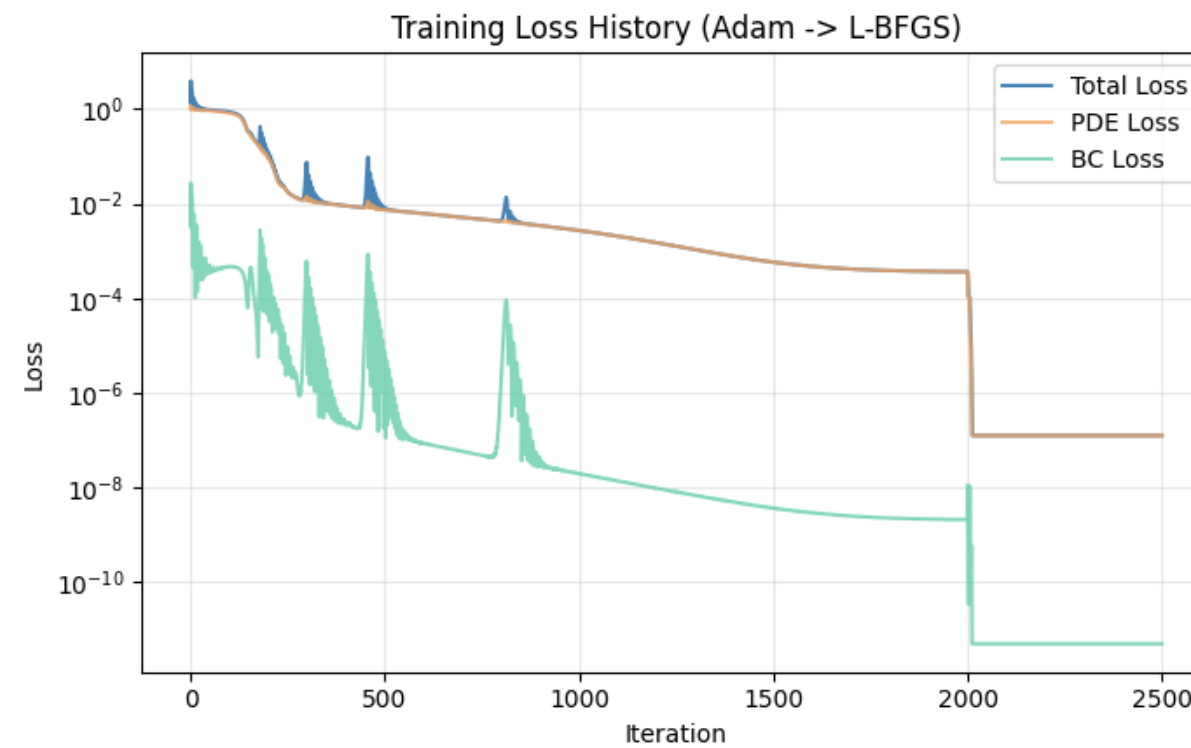
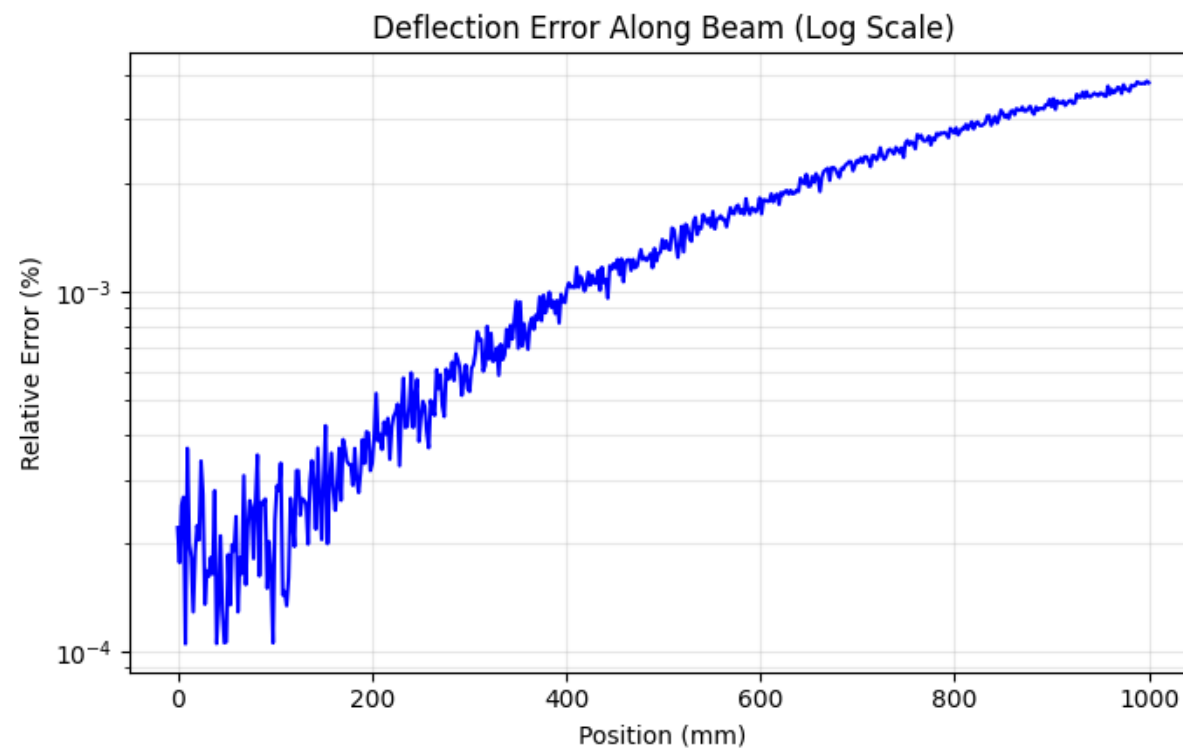
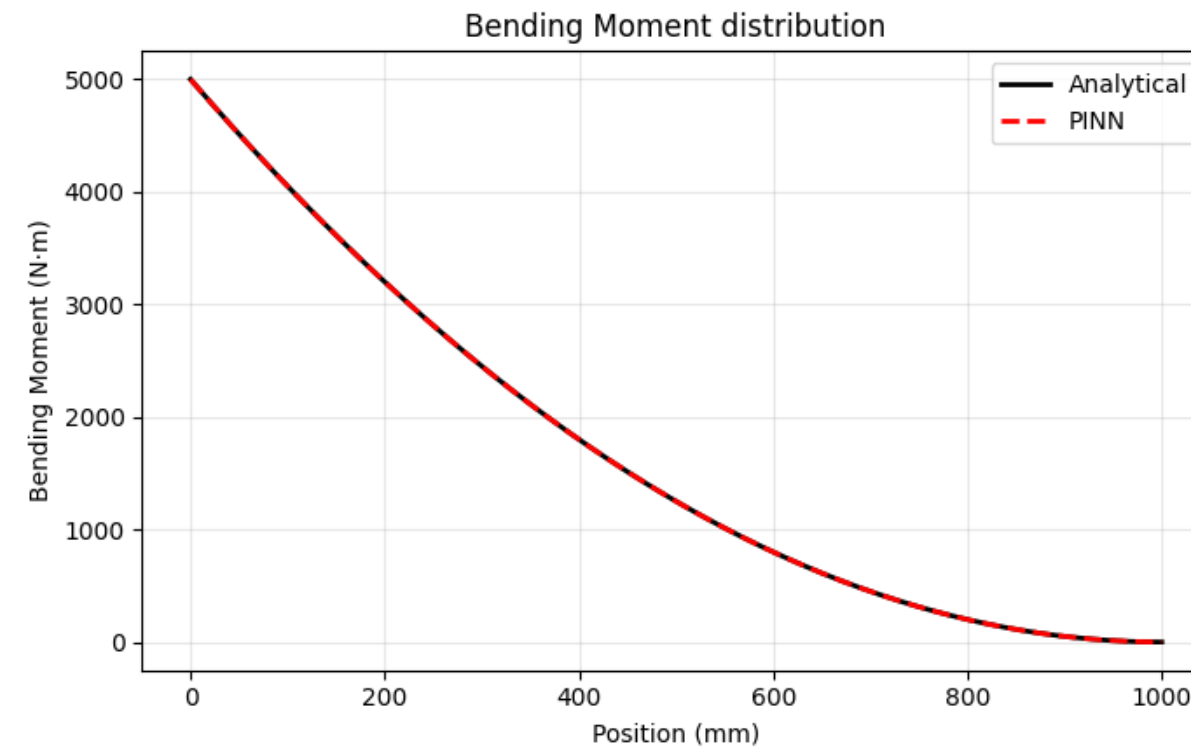
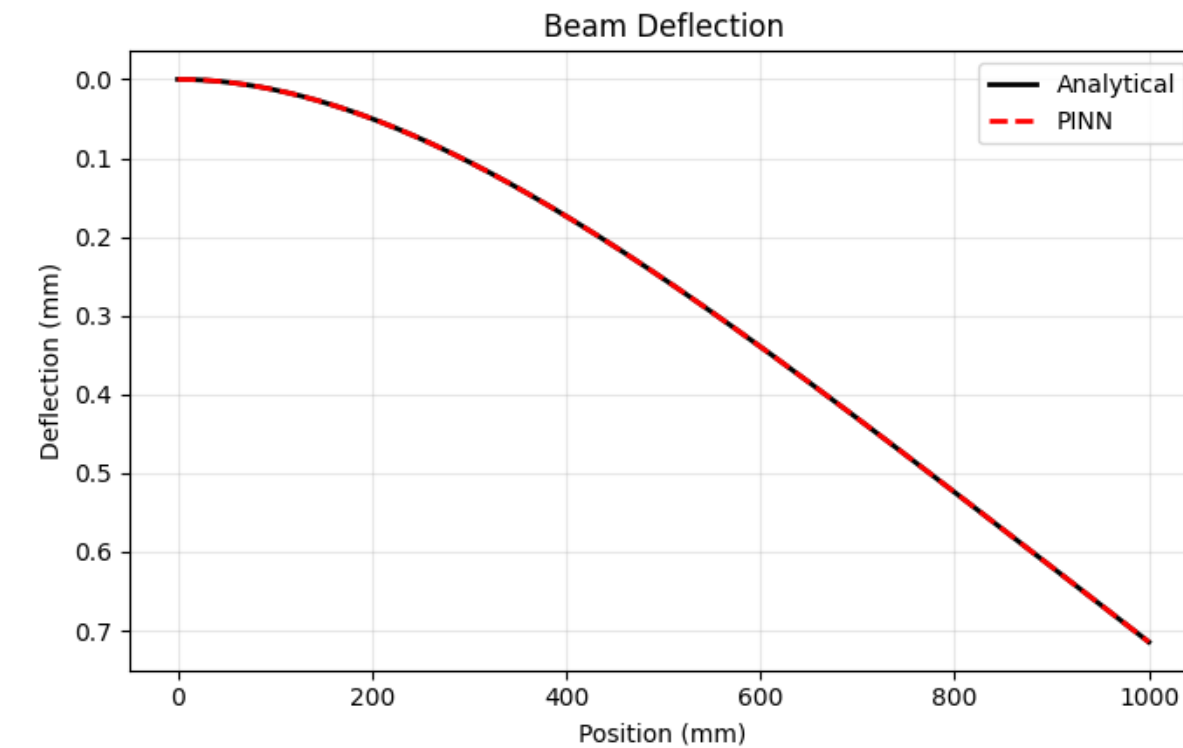
for step in range(n_lbfgs):
    def closure():
        optimizer2.zero_grad()
        loss, lpde, lbc = forward_loss(model, x_bar, lambda_bc)
        loss.backward()
        return loss

    loss = optimizer2.step(closure)
    # 重新計算以記錄各種損失分量
    _, lpde, lbc = forward_loss(model, x_bar, lambda_bc)

    loss_history["total"].append(loss.item())
    loss_history["pde"].append(lpde.item())
    loss_history["bc"].append(lbc.item())
```

4-3 : 2D Cantilever Beam with UDL-Result

Cantilever Beam PINN — Forward Problem Results



Adam 0 | Loss=1.372e+00
Adam 500 | Loss=8.108e-03
Adam 1000 | Loss=2.698e-03
Adam 1500 | Loss=5.764e-04
Forward PINN Training
(L-BFGS)...

Forward Tip Deflection Error
= 0.0038%

4-4 : Inverse Problem

Forward: Given loads + material properties → predict deflection

Inverse: Given measured deflections → discover Young's modulus E

在真實工程中，常面臨「已知幾處感測器讀數，想知道材料是否劣化（E值下降）」的情況
反問題將 $\log_E$ 設為網路內部的可訓練參數，透過物理規律與數據點共同優化來逼近真實的 E 值

$$\text{Governing Equation : } EI \frac{d^4 w}{dx^4} = q \rightarrow \frac{d^4 \bar{w}}{d\bar{x}^4} = \frac{E_{ref}}{E}$$

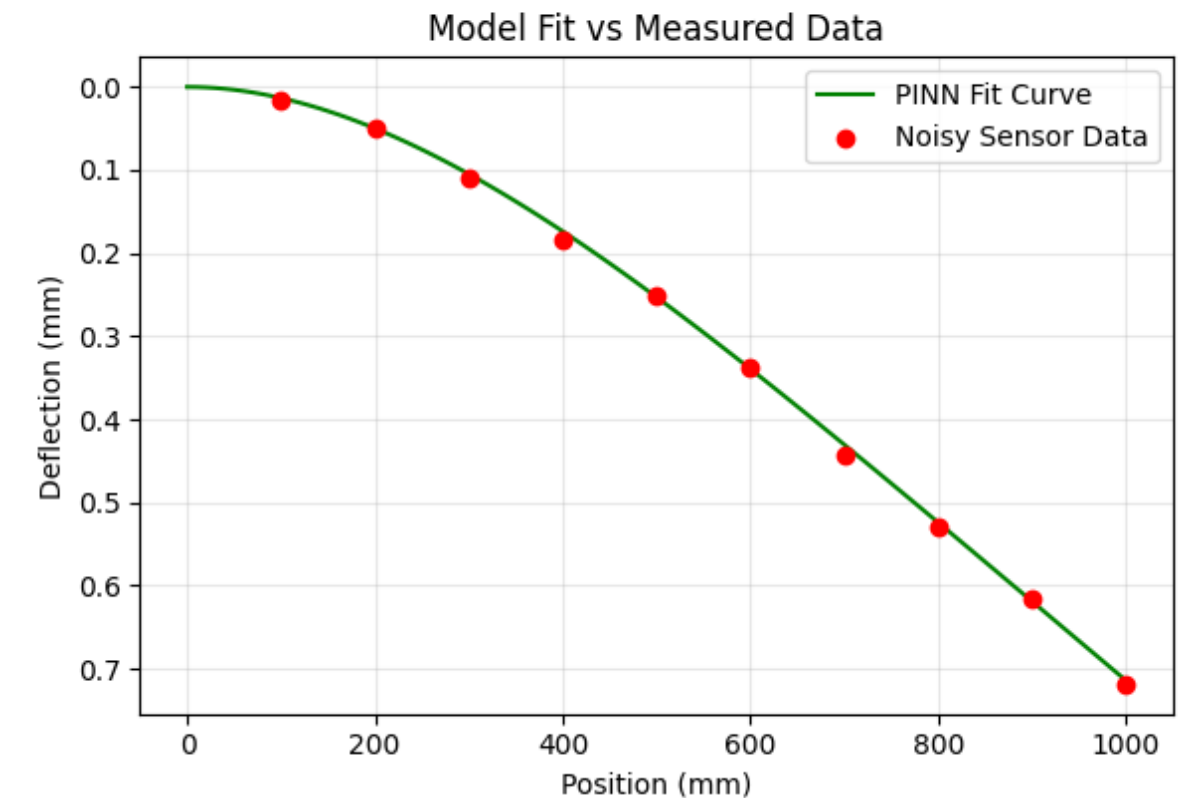
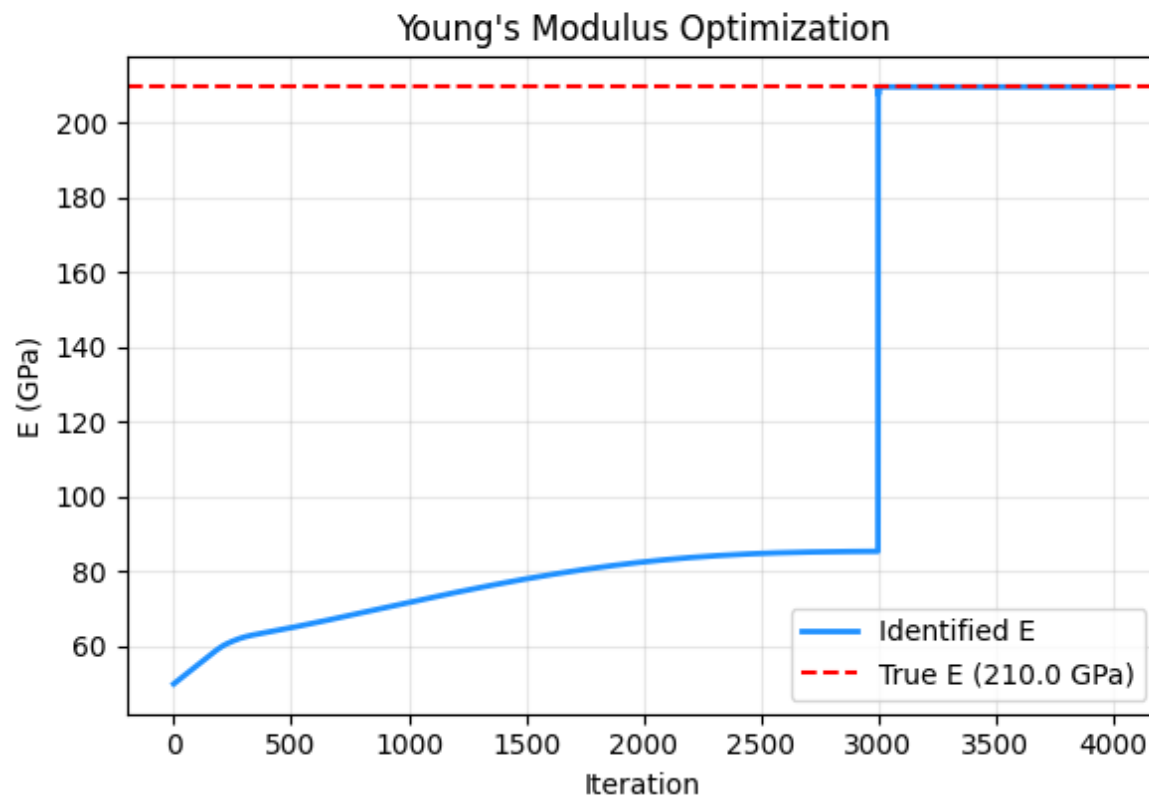
$$\text{Loss Function: Total Loss} = \text{Loss}_{PDE} + \lambda_{bc} \times \text{Loss}_{bc} + \lambda_{Data} \text{Loss}_{Data}$$

Collection Points : Sensors with 1% random noise

Epoches : Adam → 3000 Iter ; L-BFGS → 1000 Loop

4-4 : Inverse Problem-Result

Young's Modulus Parameter Identification — Inverse Problem Results

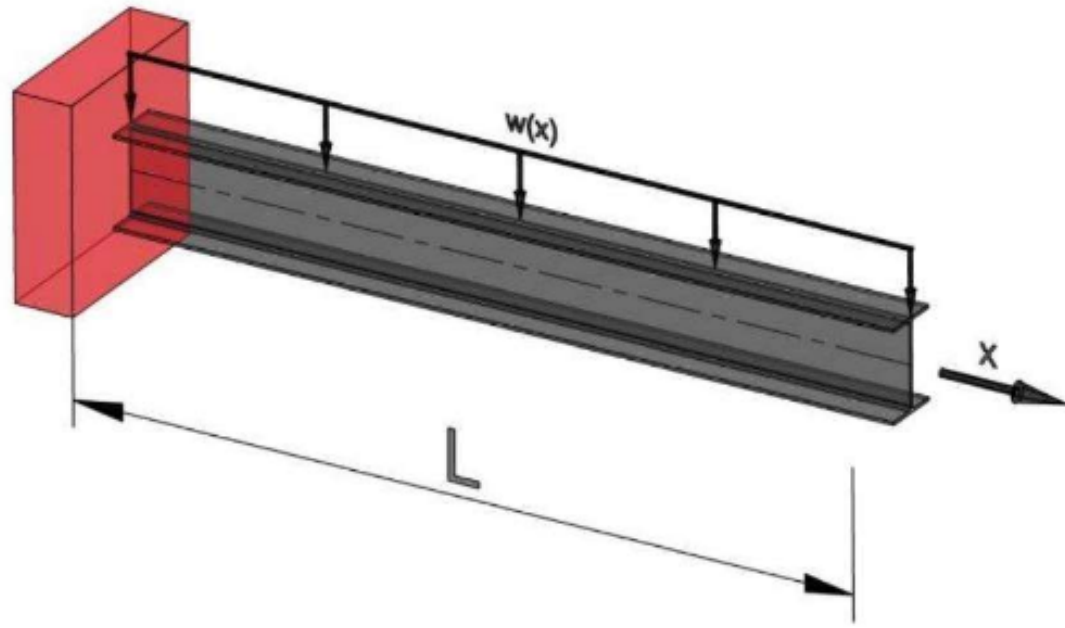


Inverse PINN Training (Adam)...

Adam 0		E = 50.00 GPa
Adam 500		E = 65.03 GPa
Adam 1000		E = 71.77 GPa
Adam 1500		E = 78.08 GPa
Adam 2000		E = 82.60 GPa
Adam 2500		E = 84.88 GPa

Discovered Young's Modulus E = 209.58 GPa
True Young's Modulus E = 210.00 GPa

4-5 : Deep Energy Method (DEM)



Governing Equation : $\Pi(u) = \mathcal{U}(u) - \mathcal{W}(u) \rightarrow \text{Minimize}$

$$U_{\text{density}} = \frac{1}{2} \lambda_{ps} (\epsilon_{xx} + \epsilon_{yy})^2 + \mu (\epsilon_{xx}^2 + \epsilon_{yy}^2 + 2 \epsilon_{xy}^2)$$

Loss Function: $\text{Loss} = U_{\text{internal}} - W_{\text{external}}$

Boundary Condition (Ansatz) : $u_x(0, y) = 0, \quad u_y(0, y) = 0$

材料參數與模型參數	
長度(L)	10.0 m
高度(h)	1.0 m
厚度(b)	1.0 m (plane stress)
Young's Modulus(E)	25GPa (concrete)
Poisson's Ratio(v)	0.3
UDL(w)	-800000.0 N/m
Training Points	Nx=200, Ny=20, Total:4000
Top Boundary Nodes	200
學習率(Lr)	0.01
優化演算法	Torch.optim.LBFGS (iter=50)
Epochs	300

4-5 : Deep Energy Method (DEM)

(1)Network

```
class MultiLayerNet(torch.nn.Module):
    def __init__(self, D_in, H, D_out):
        super(MultiLayerNet, self).__init__()

        # 用 torch.nn.Sequential 封裝成 3 層隱藏層的深層網路
        self.net = torch.nn.Sequential(
            torch.nn.Linear(D_in, H),          # 輸入層 (2D 座標) -> 隱藏層 1 (H)
            torch.nn.Tanh(),                    # 激活函數 1
            torch.nn.Linear(H, H),              # 隱藏層 1 (H) -> 隱藏層 2 (H)
            torch.nn.Tanh(),                    # 激活函數 2
            torch.nn.Linear(H, H),              # 隱藏層 2 (H) -> 隱藏層 3 (H)
            torch.nn.Tanh(),                    # 激活函數 3
            torch.nn.Linear(H, D_out)           # 隱藏層 3 (H) -> 輸出層 (2D 位移 ux, uy)
        )
```

MLP with activation function tanh()

Responsible for simulating continuous displacement field

4-5 : Deep Energy Method (DEM)

(2)Boundary Condition

```
def getU(self, x):  
    u = self.model(x)  
    Ux = x[:, 0] * u[:, 0]    # 固定端邊界條件 (x=0 時 ux=0)  
    Uy = x[:, 0] * u[:, 1]    # 固定端邊界條件 (x=0 時 uy=0)  
    return torch.stack([Ux, Uy], dim=-1)
```

傳統 PINNs 是把邊界條件當成懲罰項（Loss Penalty）塞進 Loss 裡，網路容易顧此失彼
這段程式碼採用了 **Ansatz** 技巧，透過令 $Ux = x \cdot NNx$ ，不論神經網路怎麼亂猜，只要帶入 $x=0$ ，
位移就絕對是 0。

這在物理上滿足了懸臂梁緊緊嵌在牆壁上的真實狀態

4-5 : Deep Energy Method (DEM)

(3)Physics Loss

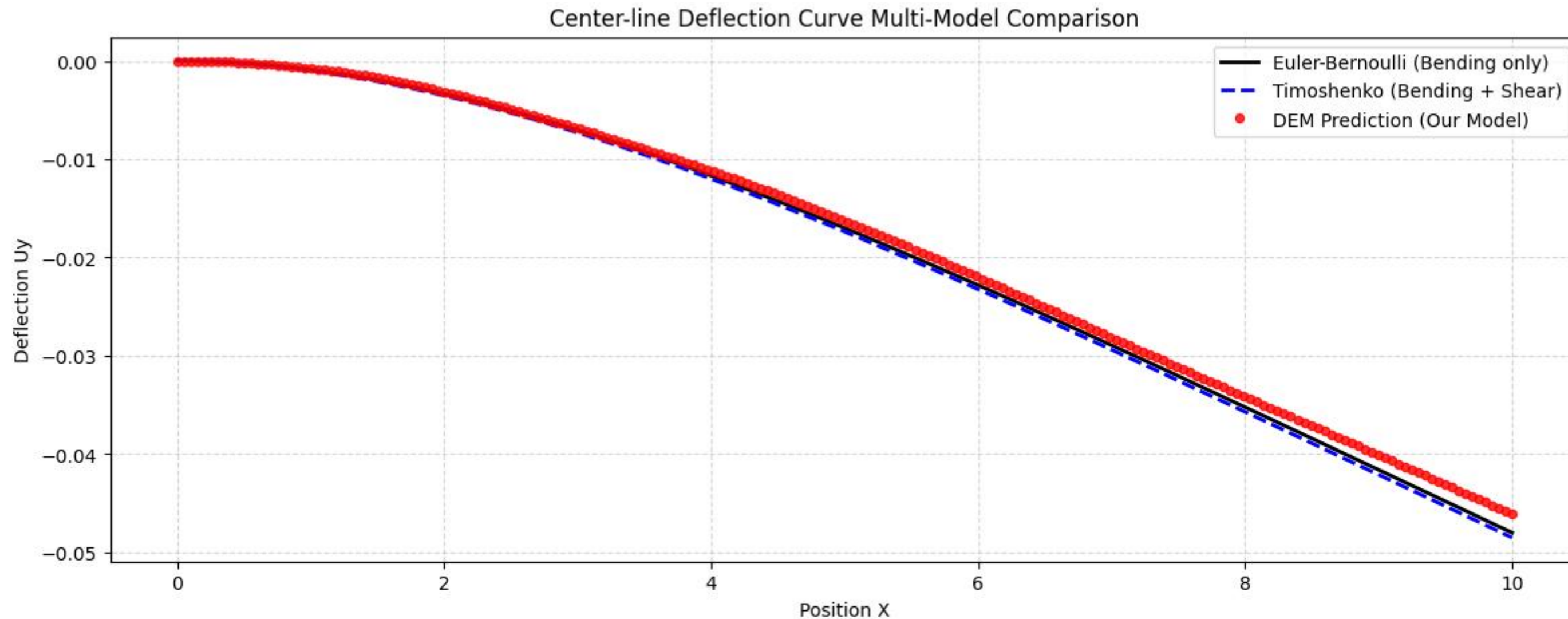
```
# 【1. 內部總應變能積分】
# 將應變能密度的「平均值」乘以梁的「總體積」，以黎曼和的概念模擬全域體積分。
U_internal = torch.mean(storedEnergy) * (cf.Length * cf.Height * cf.Thickness)

# 【2. 均布載荷外力功積分】
u_top = self.getU(x_top)
# 計算上表面 Y 方向垂直位移的平均值，乘以垂直均布載荷 w_load，再乘以受力長度 Length，模擬線積分。
W_external = torch.mean(u_top[:, 1]) * cf.w_load * cf.Length

# 【3. 計算總位能損失】損失函數 = 內部總應變能 - 外力功。
loss = U_internal - W_external
optimizer.zero_grad()
loss.backward()
return loss
```

$$\underbrace{\mathcal{L}(\boldsymbol{\theta})}_{\text{Potential energy loss}} = \underbrace{\int_B \Psi(\hat{\mathbf{F}}) dV}_{\text{Internal energy NN}} - \underbrace{\left(\int_B \mathbf{f}_b \cdot \hat{\boldsymbol{\varphi}}(X; \boldsymbol{\theta}) dV + \int_{\partial B_t} \bar{\mathbf{t}} \cdot \hat{\boldsymbol{\varphi}}(X; \boldsymbol{\theta}) dA \right)}_{\text{External energy NN}},$$

4-5 : Deep Energy Method (DEM)-Results 1.



兩種均布載重驗證結果

Euler Beam 理論最大撓度 (僅考慮彎曲) : -0.048000

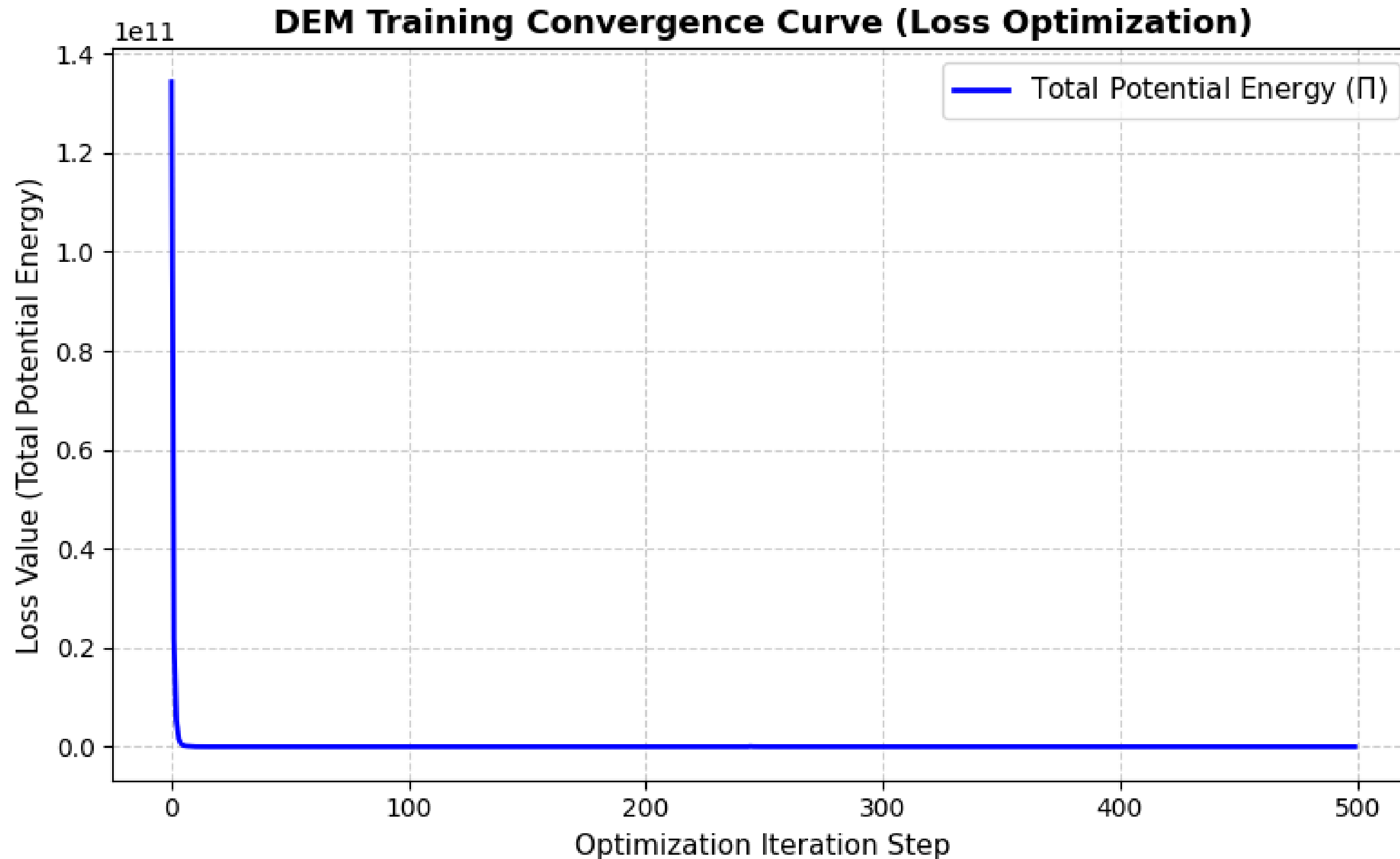
Timoshenko Beam 理論最大撓度 (加上考慮剪切) : -0.048499

深度能量法(DEM)預測最大撓度 : -0.046111

DEM 相對於「Euler Beam」之誤差 : 3.94 %

DEM 相對於「Timoshenko Beam」之誤差 : 4.92 % (物理修正後)

4-5 : Deep Energy Method (DEM)-Results 2.



Part 5 : Conclusion

What We Have Learned & Future

Challenge

- In this project, we explored the application of AI in civil engineering.
- PINNs performed well in solving forward and inverse problems of one-dimensional and two-dimensional linear elastic structural materials.
- For the loss function, the deep energy method avoids boundary condition oscillations and describes the problem with a simpler, more realistic physics-based domain equation.
- We hope to further refine this method and apply it to complex geometries and extreme environments to overcome the limitations of current finite element methods.